

CAREERBOOST AI: RESUME EVALUATION AND LEARNING ASSISTANT USING MACHINE LEARNING

SRIKANTH R
SUGASHINI S
SHRUTHI S

MINI PROJECT REPORT

Submitted to the

SCHOOL OF ENGINEERING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)
CHENNAI – 600 117

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "**CAREERBOOST AI: RESUME EVALUATION AND LEARNING ASSISTANT USING MACHINE LEARNING**", is the Bonafide work of **SRIKANTH R, SUGASHINI S and SHRUTHI S** who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD
Dr. K. Ulaga Priya M.E., Ph.D
Associate Professor & Head
Department of AI&ML
School of Engineering
VISTAS, Chennai

SIGNATURE OF THE SUPERVISOR
Ms. Divya Bairavi.S, M.E. (PhD)
Assistant Professor
Department of AI&ML
School of Engineering
VISTAS, Chennai

Submitted for the Project Work and Viva-Voce examination held on ..... at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support we received from the positive minds around us in making this venture a successful one. The infrastructural support with all kinds of lab facilities has been a motivating factor in the completion of this project work, all because of our **FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH**, and it is with great pleasure that we take this opportunity to thank him.

We are grateful to our **HEAD OF THE DEPARTMENT, Dr. K. Ulaga Priya**, for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through ongoing advice and encouragement to this day. We extend our warmest thanks to our **INTERNAL GUIDE, Ms. Divya Bairavi.S, Assistant Professor, Department of AI&ML**, who has patiently guided us and provided invaluable advice and help.

We would like to express our greatest gratitude to our staff members, the Librarian and the non-teaching staff members of the Computer Centre, who have helped and supported us throughout. Last but not least, we wish to thank our parents and friends for their individual support and interest, which inspired and encouraged us to complete this project.

SRIKANTH R
SUGASHINI S
SHRUTHI S

TABLE OF CONTENTS

CHAPTER NO.	TABLE OF CONTENT	PAGE NO.
	ABSTRACT	**i**
	LIST OF FIGURES	**ii**
	LIST OF TABLES	**iii**
	ABBREVIATIONS	**iv**
1	**INTRODUCTION**	**1**
	1.1 OVERVIEW OF THE PROJECT	1
	1.2 SCOPE AND OBJECTIVE	4
2	**LITERATURE REVIEW**	**8**
	2.1 LITERATURE OVERVIEW	8
	2.2 LITERATURE SURVEY	8
3	**SYSTEM DESIGN**	**12**
	3.1 EXISTING SYSTEM	12
	3.2 PROPOSED SYSTEM	12
	3.3 SYSTEM ARCHITECTURE	14
	3.4 HARDWARE REQUIREMENTS	20
	3.5 SOFTWARE REQUIREMENTS	20
4	**IMPLEMENTATION**	**22**
	4.1 METHODOLOGY	22
	4.2 IMPLEMENTATION	25
	4.2.1 Resume Upload and Parsing Module	25
	4.2.2 Text Preprocessing Module	26
	4.2.3 Information Extraction Module	26
	4.2.4 Skill Matching and Gap Analysis Module	26
	4.2.5 Recommendation and Dashboard Module	26
	4.3 DATABASE DESIGN AND IMPLEMENTATION STATUS	27
	4.4 SECURITY AND PRIVACY CONSIDERATIONS	27
5	**CONCLUSION AND FUTURE WORK**	**28**
	5.1 CONCLUSION	28
	5.2 FUTURE WORK	29
	5.3 SOURCE CODE	29
	5.4 RESULTS	37
	REFERENCES	**44**

ABSTRACT

Many students and fresh graduates struggle to understand whether their resume truly reflects the skills expected in their target career, often causing them to miss internship and job opportunities. Existing resume-checking tools mostly focus on grammar, formatting or keyword matching, and rarely offer personalised career direction. This project proposes CareerBoost AI, a web-based Resume Evaluation and Learning Assistant that uses Natural Language Processing (NLP) and Machine Learning to analyse uploaded resumes in PDF or DOCX format. The system extracts details such as education, technical skills, projects, certifications and experience, then compares the resume with a career domain such as Data Science, Web Development, Full Stack Development or Cybersecurity.

Based on the domain, the system calculates a skill-match score from weighted skill coverage and TF-IDF cosine similarity, with an optional Sentence-BERT semantic layer; identifies missing skills; and recommends relevant courses, certifications and learning platforms, along with suggestions to make the resume more attractive to recruiters. A working Python prototype built with open-source libraries was tested on sample PDF and DOCX resumes, passing all thirteen functional tests with an average analysis time of a few tens of milliseconds. Future enhancements include a supervised domain classifier, integration with live job-market data, multilingual resume support and compatibility with major job portals.

Keywords: Natural Language Processing (NLP); Machine Learning; Resume Analysis; Career Recommendation; Skill Gap Analysis; TF-IDF; Sentence-BERT; Artificial Intelligence.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	System architecture of CareerBoost AI	14
3.2	Processing pipeline of CareerBoost AI	15
3.3	Data flow diagram, Level 0 (context diagram)	16
3.4	Data flow diagram, Level 1	16
3.5	Use case diagram	17
3.6	Flowchart of the resume analysis process	18
3.7	Sequence diagram of a resume analysis request	19
3.8	Entity-relationship diagram	20
4.1	Skill-match scoring model	24
5.1	Match score of each sample resume against each domain	39
5.2	Matched and missing skills for the suggested domain	40
5.3	Dashboard output for Resume A (Data Science)	40
5.4	Dashboard output for Resume B (Web Development ranked first)	41
5.5	Dashboard output for Resume C (Cybersecurity)	41

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Summary of the literature survey	10
3.1	Existing systems versus the proposed CareerBoost AI	13
3.2	Hardware requirements	20
3.3	Software requirements	20
4.1	Skill taxonomy used by the prototype	23
5.1	Sample resumes used for the demonstration	38
5.2	Skills extracted from each resume	38
5.3	Score components for the suggested domain	39
5.4	Learning resources recommended for Resume C	42
5.5	Functional test cases and results	42

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ATS	Applicant Tracking System
BERT	Bidirectional Encoder Representations from Transformers
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DOCX	Office Open XML Word Document
ER	Entity-Relationship
GPU	Graphics Processing Unit
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
IDF	Inverse Document Frequency
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
NLTK	Natural Language Toolkit
OCR	Optical Character Recognition
PDF	Portable Document Format
RAM	Random Access Memory
REST	Representational State Transfer
SBERT	Sentence-BERT
SQL	Structured Query Language
TF-IDF	Term Frequency–Inverse Document Frequency
UI	User Interface

CHAPTER 1. INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Introduction

A resume is the first document a recruiter reads and, for most students and fresh graduates, the only evidence of their skills at the start of a career. It summarises education, technical skills, projects, certifications and experience in a few pages. Yet many students cannot tell whether their resume actually reflects the skills expected in the career they are aiming for. As a result they miss internship and job opportunities without ever learning which skills were absent.

Existing resume-checking tools mostly concentrate on grammar, formatting or keyword matching. Recruiter-oriented systems rank candidates against a job description, but they rarely tell the candidate what to learn next [5]–[7]. Course-recommendation research, on the other hand, usually starts from a student's declared interests and not from the content of the resume itself [3]. There is therefore a practical gap between resume evaluation and personalised learning guidance.

This project presents **CareerBoost AI**, a web-based Resume Evaluation and Learning Assistant. It uses Natural Language Processing (NLP) and Machine Learning (ML) to read an uploaded resume in PDF or DOCX format, extract its key entities, compare the extracted skills with the requirements of a career domain, calculate a skill-match score, identify missing skills and recommend courses, certifications and learning platforms to close the gaps. Simple suggestions for making the resume more attractive to recruiters are provided alongside.

Problem Statement

Students and job seekers often struggle to identify the skills missing from their resumes for a desired career. Conventional screening methods rely on formatting checks and keyword matching, and they have several weaknesses:

- **No personalised direction:** A keyword score does not explain which skills are missing or which resources would build them.
- **Vocabulary mismatch:** The same skill is written in many ways (for example "sklearn", "scikit-learn" and "scikit learn"), which simple keyword rules miss [2].
- **Recruiter-side focus:** Most tools are built to help employers filter candidates, not to help candidates improve [5], [6].
- **Unclear career fit:** A student with skills in several areas has no objective way to see which domain the resume currently suits best.

The core problem is the lack of an accessible system that connects resume content, career-domain requirements and learning recommendations in a single workflow.

Objectives of the Project

The main objectives of this project are:

- To develop CareerBoost AI for analysing resumes using NLP and Machine Learning techniques.
- To extract skills, education, projects, experience and certifications from a resume.
- To compare the resume with the requirements of a selected job role or career domain.
- To find the matched skills and the missing skills.
- To calculate a skill-matching score for the selected job role.
- To suggest courses and learning resources for the missing skills.
- To provide simple suggestions for improving the resume.

Scope of the Project

CareerBoost AI is designed as an assistive tool for students, graduates and job seekers, not as a replacement for human career counsellors. Its scope includes:

- **Resume Input:** Upload of resumes in PDF or DOCX format.
- **Domain Coverage:** Four career domains in the current taxonomy: Data Science, Web Development, Full Stack Development and Cybersecurity.
- **Analysis:** Extraction of entities, skill matching, gap analysis and score calculation.
- **Guidance:** Course, certification and learning-platform recommendations, and resume-improvement tips.
- **Presentation:** A web dashboard that summarises the resume, the skill match, the gaps and the recommendations.

System Overview

The system integrates four major components:

1. **Resume Parsing and Preprocessing:** Text is extracted from the PDF or DOCX file, split into sections and cleaned by tokenisation, stop-word removal and lemmatisation.
2. **Information Extraction:** Skills, education, projects, certifications and experience are identified.
3. **Matching and Scoring Engine:** The extracted skills are compared with the required skills of the selected domain using weighted skill coverage, TF-IDF and cosine similarity, with an optional Sentence-BERT semantic layer.
4. **Recommendation Engine and Dashboard:** Learning resources are suggested for each missing skill and all results are shown on a dashboard.

Advantages of the System

- **Personalised guidance:** The output is specific to the individual resume and the chosen domain.
- **Transparency:** The score is built from visible components (matched skills, missing skills and similarity), so the user can see why a score was given.
- **Actionable output:** Every missing skill is linked to at least one learning resource.
- **Open-source and low cost:** The system is built with open-source libraries and requires no paid software.
- **Scalable design:** New domains, skills and courses are added by editing the skill taxonomy and the resource catalogue.

Applications of the Project

- **Students and fresh graduates:** Self-assessment before applying for internships and jobs.
- **College placement cells:** Quick, consistent feedback to large batches of students.
- **Career-guidance and training institutes:** Identification of common skill gaps in a batch to plan training.
- **Job portals:** A candidate-side feature that suggests what to learn for a target role.

Expected Outcomes

At the end of this project the following outcomes are expected:

- A working prototype that analyses PDF and DOCX resumes and identifies relevant skills and qualifications.
- A skill-match score and a matched/missing-skill report for a chosen career domain.
- Personalised resume-improvement suggestions.
- Course and resource recommendations based on the identified skill gaps.
- Improved career readiness and job opportunities for students and job seekers.

Future Enhancements

Future versions of the system could include integration with live job-market data, multilingual resume support and compatibility with major job portals. A supervised domain classifier and a fine-tuned semantic model can be added once a labelled resume dataset has been collected.

1.2 SCOPE AND OBJECTIVE

Purpose of the Study

The main purpose of this project is to design, implement and test an intelligent resume evaluation and learning assistant that helps students understand how well their resume matches a chosen career domain and what they should learn to improve. This is achieved by:

1. Analysing the content of the resume with NLP techniques to obtain a structured profile.
2. Comparing that profile with domain requirements using machine-learning-based similarity measures.
3. Converting the identified gaps into concrete learning recommendations.

General Objective

The general objective is to create a scalable, open-source, AI-based system that reads a resume, evaluates it against a career domain, reports the skill match and skill gaps and guides the user towards suitable courses and certifications, so that career readiness improves.

Specific Objectives

The specific objectives required to achieve this goal are:

- To build a parser that reliably extracts text from PDF and DOCX resumes and rejects unsupported or unreadable files.
- To design an NLP preprocessing pipeline consisting of cleaning, tokenisation, stop-word removal and lemmatisation.
- To extract skills while handling spelling variants and synonyms.
- To design a weighted skill taxonomy for each supported domain, separating core skills from optional skills.
- To calculate a transparent skill-match score from skill coverage and TF-IDF cosine similarity.
- To build a recommendation module that maps each missing skill to learning resources.
- To present the results on a web dashboard and to validate the prototype with functional tests and response-time measurements.

Scope of the Project

The scope defines the functionalities, technical boundaries and applications of the system.

A. Functional Scope

- **Resume Upload:** PDF and DOCX files up to 5 MB.
- **Domain Selection:** The user selects a domain, or the system suggests the best-fitting domain from the ranking of all domains.

- **Analysis Output:** Resume summary, skill-match score, matched and missing skills, course recommendations and improvement tips.

B. Technical Scope

- **Frontend:** HTML, CSS, JavaScript and Bootstrap, with Chart.js for visualisation.
- **Backend:** Python with the Flask framework.
- **NLP:** NLTK and spaCy for text processing; PyMuPDF and python-docx for parsing.
- **Machine Learning:** Scikit-learn for TF-IDF and cosine similarity; Sentence-BERT for optional semantic matching.
- **Database:** MySQL for users, resumes, extracted information, skills, courses and analysis logs.

C. Application Scope

The project is applicable wherever resumes must be assessed against skill requirements: colleges and placement cells, training institutes, career-guidance portals and job-search platforms.

Importance of the Project

This project addresses the missing link between resume screening and skill development. It focuses not only on what a resume contains but on what it lacks for a particular career. Its key benefits are:

- **Early awareness:** Students learn about skill gaps before they apply, not after rejection.
- **Focused learning:** Recommendations target the missing skills, saving time compared with generic course lists.
- **Fairness and consistency:** Every resume is evaluated with the same transparent rules.
- **Support for institutions:** Placement cells obtain a consistent, data-based view of batch readiness.

System Functionality Overview

The system performs the following key operations:

1. **Resume Upload:** The user uploads a PDF or DOCX file and optionally selects a domain.
2. **Parsing:** Text is extracted and divided into sections such as education, skills, projects and certifications.
3. **Preprocessing:** The text is cleaned, tokenised, stripped of stop-words and lemmatised.
4. **Information Extraction:** Contact details, degrees and skills are identified.
5. **Matching and Scoring:** Skills are compared with the domain profile and a score between 0 and 100 is calculated.

6. **Recommendation:** Learning resources and resume tips are generated.
7. **Dashboard:** Results are displayed to the user.

Methodology for Achieving Objectives

The project follows a modular development approach divided into the following stages:

1. **Requirement Analysis:** Understanding user needs, supported domains and the information a resume contains.
2. **Taxonomy Design:** Defining core and optional skills and synonyms for each domain.
3. **Module Development:** Building the parser, preprocessor, extractor, matcher and recommender.
4. **Integration:** Connecting the modules through a Flask web application.
5. **Testing:** Verifying behaviour with functional test cases and measuring response time.
6. **Documentation and Demonstration:** Preparing the report and demonstrating the prototype.

Benefits of the Proposed System

1. **Better career readiness:** Users know exactly what to improve.
2. **Time saving:** Analysis completes in a fraction of a second on a standard laptop (Chapter 5).
3. **Cross-platform access:** The dashboard runs in any modern browser.
4. **Extensibility:** New domains and courses can be added without changing the code.
5. **Low cost:** Only open-source components are used.

Challenges and Limitations

Despite its strengths, the system has certain limitations:

- Skill extraction depends on the coverage of the skill taxonomy and its synonym list; skills outside the taxonomy are not recognised.
- Scanned (image-only) resumes need OCR, which is not part of the current prototype.
- Unusual layouts such as multi-column designs may alter the order of extracted text.
- Closely related domains, such as Web Development and Full Stack Development, share many skills and may receive similar scores.
- Resume data is personal; privacy and secure handling must be ensured in real deployments.

Organisation of the Report

Chapter 1 introduces the problem, objectives and scope. Chapter 2 reviews related work. Chapter 3 describes the existing and proposed systems, the architecture and the hardware and

software requirements. Chapter 4 explains the methodology and the implementation of each module. Chapter 5 presents the conclusion, future work, source code and results, followed by the references.

CHAPTER 2. LITERATURE REVIEW

2.1 LITERATURE OVERVIEW

Automated analysis of resumes has grown rapidly with the spread of online recruitment. Early systems extracted fields from resumes with hand-written rules and regular expressions. Later systems represented resumes and job descriptions as term vectors and ranked them with similarity measures. More recently, transformer-based language models have made it possible to compare texts by meaning and not only by shared words [2], [9], [10].

Most of this research is written from the recruiter's point of view: given many resumes and one vacancy, which candidates should be shortlisted? A second, smaller body of work looks at the candidate's point of view, including career-counselling systems and course recommenders that help students choose what to study [3], [4]. CareerBoost AI sits between these two streams. It analyses the content of the candidate's own resume, as screening tools do, but it uses the result to recommend learning, as career-guidance tools do.

This literature review serves the following purposes:

- To understand how existing systems parse, represent and match resumes.
- To compare keyword-based, similarity-based and transformer-based approaches.
- To identify the research gap that CareerBoost AI aims to fill.
- To provide the theoretical foundation for the techniques used in this project.

2.2 LITERATURE SURVEY

The survey is organised into thematic sub-sections, from resume screening to semantic matching and career guidance. Table 2.1 summarises the principal studies reviewed.

Resume Screening and Analysis Tools

Abhishek, Niranjanamurthy et al. developed an AI-powered resume analysis and recommendation tool that analyses all resume components, gives applicants personalised predictions and recommendations for improving their resumes, and exports results to CSV for recruiters [1]. Sabarirajan et al. proposed an AI-based screening and ranking framework that uses deep learning and NLP to shortlist resumes at scale, with attention to fairness and explainability [5]. Tijare et al. built a smart screening tool that retrieves and processes company resumes automatically and reported an average accuracy of more than 90 %; the authors note that it should be extended to resumes written in languages other than English [6]. These systems demonstrate that NLP can automate resume reading, but their main users are recruiters.

Similarity-Based Matching

Daryani et al. described a two-phase screening system that parses qualification details with NLP and then ranks candidates by comparing applicant profiles with job requirements using a

vector space model and content-based recommendation [7]. The vector space model rests on term weighting. Spärck Jones introduced inverse document frequency to weight rare terms more heavily [14], and Salton and Buckley compared term-weighting schemes for text retrieval, establishing the TF-IDF family of weights [13]. Cosine similarity between TF-IDF vectors is a standard way to measure how close two documents are [15]. CareerBoost AI uses the same foundations for its domain-similarity component.

Transformer-Based Semantic Matching

Keyword methods cannot match texts that use different words for the same idea. Transformer models address this. Devlin et al. introduced BERT, which learns contextual language representations [10], building on the attention mechanism of Vaswani et al. [11]. Reimers and Gurevych proposed Sentence-BERT, which uses Siamese networks to produce sentence embeddings that can be compared directly with cosine similarity, making large-scale semantic comparison practical [9]. Wang et al. later introduced MiniLM, a compressed transformer from which compact sentence-embedding models are derived [12].

Lavi et al. applied this idea to recruitment. They fine-tuned a Siamese Sentence-BERT model, called conSultantBERT, on more than 270,000 resume-vacancy pairs labelled by staffing consultants. The model outperformed baselines built on TF-IDF features and pre-trained BERT embeddings, and it also matched multilingual and cross-lingual text [2]. Its limitation for student projects is its dependence on a very large proprietary labelled dataset.

Career Guidance and Course Recommendation

Parab et al. built a career-counselling bot that analyses a user's questions with AI algorithms and answers career-related queries such as the best field to choose or trending courses [4]. Such a bot depends on the questions the user asks and does not read the user's own resume. Soni, Suleiman and Singh designed a course-recommendation system for university students that uses data mining and collaborative filtering to relate past courses and career goals to recommended courses, with a front-end interface refined through user feedback [3]. Lai et al. presented CareerMapper, an automated tool that reviews a user's online profile and recommends improvements by analysing a large number of other profiles [8]. These studies show the value of guidance for candidates, but none of them combines resume parsing, domain-specific skill-gap analysis and learning-resource recommendation in one open and explainable workflow.

NLP Toolkits Used in Resume Analysis

Practical systems are built on mature open-source toolkits. NLTK provides tokenisation, stop-word lists and lemmatisation [16]; spaCy offers industrial-strength pipelines including named-entity recognition [17]; Scikit-learn supplies TF-IDF vectorisation, similarity functions and

classifiers [18]; and Flask is a lightweight Python framework well suited to serving such models through a web interface [19]. CareerBoost AI is assembled from these components.

Table 2.1: Summary of the literature survey

Ref.	Study	Methodology	Data / basis	Limitation with respect to this project
[1]	Abhishek, Niranjanamurthy et al., 2025	NLP-based resume analysis, personalised prediction and recommendation, CSV export	Uploaded resumes	Oriented to resume analysis and recruiter analytics; learning resources for skill gaps are not its focus
[2]	Lavi, Medentsiy and Graus, 2021	Siamese Sentence-BERT fine-tuned with a cosine-similarity objective	270,000+ labelled resume-vacancy pairs (Randstad)	Needs large proprietary labelled data; not reproducible with public data
[3]	Soni, Suleiman and Singh, 2025	Data mining and collaborative filtering for course recommendation	University course and career-interest data	Based on declared interests, not on resume content
[4]	Parab, Palkar, Maurya and Balpande, 2017	AI-based chatbot analysing user queries	Question-answer knowledge	Query-driven; does not analyse the user's resume
[5]	Sabarirajan et al., 2025	Deep learning and NLP for screening and ranking with fairness focus	Resume and job-description data	Recruiter-side ranking; no candidate learning plan
[6]	Tijare et al., 2023	Automatic retrieval and processing of resumes for shortlisting	Company resumes	Recruiter-side; English resumes only
[7]	Daryani et al., 2020	NLP parsing, vector space model, content-based similarity	Resumes and job descriptions	Ranks candidates; does not identify skill gaps
[8]	Lai et al., 2016	Profile evaluation using analysis of many online profiles	Online (LinkedIn) profiles	Profile-level advice, not domain-specific skill gaps with courses

Research Gaps Identified

From the review, the following gaps emerge:

- **Recruiter-centred design:** Most resume systems rank candidates for employers and give little help to the candidate [5]–[7].
- **Missing link to learning:** Course recommenders start from interests or past courses, not from the skills actually present in a resume [3], [4].
- **Limited explainability:** Highly accurate semantic models are costly to train and hard to interpret for students [2].
- **Dependence on large datasets:** Fine-tuned models need labelled data that student projects rarely have.

CareerBoost AI addresses these gaps with a candidate-centred, explainable pipeline that combines transparent skill-gap analysis with an optional semantic layer and links each gap to learning resources.

CHAPTER 3. SYSTEM DESIGN

3.1 EXISTING SYSTEM

Overview

Students currently evaluate their resumes in three main ways: asking seniors or teachers for feedback, using online resume checkers, or applying to jobs and waiting for responses. Online checkers typically examine formatting, spelling and the presence of keywords from a pasted job description. Applicant Tracking Systems (ATS) used by employers also rely heavily on keyword matching and give no feedback to the candidate.

Limitations of Existing Methods

- **Surface-level checks:** Grammar, layout and keyword counts say little about whether the skills match a career.
- **No skill-gap report:** The user is not told which specific skills are missing.
- **Inconsistent manual feedback:** Advice from individuals varies and does not scale to a whole batch.
- **No learning path:** Even when a weakness is noticed, no resources are suggested.
- **Vocabulary sensitivity:** Different spellings or abbreviations of the same skill are treated as different words.

Common Issues in Existing Systems

- Keyword stuffing can inflate scores without real skills.
- Results are not explained, so users cannot act on them.
- Candidate-side tools are scarce compared with recruiter-side tools.
- Career-domain suggestions are rarely provided.

3.2 PROPOSED SYSTEM

Overview

The proposed system, CareerBoost AI, is a web-based Resume Evaluation and Learning Assistant. The user uploads a resume and selects a career domain (or lets the system suggest one). The system extracts the resume text, performs NLP preprocessing, identifies skills, compares them with the required skills of the domain, calculates a skill-match score, lists the matched and missing skills and recommends courses and resources for the missing skills. The final results are displayed on a dashboard.

Objectives

- To evaluate a resume against a chosen career domain objectively and transparently.
- To report matched and missing skills in a form the user can act on.

- To recommend learning resources for each missing skill.
- To give simple, rule-based suggestions for improving the resume.

Features of the Proposed System

- **Multi-format parsing:** PDF (PyMuPDF) and DOCX (python-docx) resumes.
- **NLP preprocessing:** Cleaning, tokenisation, stop-word removal and lemmatisation.
- **Synonym-aware skill extraction:** Spelling variants map to one canonical skill.
- **Weighted skill taxonomy:** Core skills count double and optional skills count once.
- **Hybrid scoring:** Skill coverage combined with TF-IDF cosine similarity; optional Sentence-BERT semantic similarity.
- **Domain suggestion:** All domains are ranked so the user can see the best fit.
- **Learning recommendations:** Courses, tutorials, documentation and certifications for each gap.
- **Dashboard:** Resume summary, score, skill gaps and recommendations.

Comparison of Existing and Proposed Systems

Table 3.1: Existing systems versus the proposed CareerBoost AI

Aspect	Existing systems	Proposed CareerBoost AI
Primary user	Recruiter or general user	Student, graduate or job seeker
Analysis basis	Formatting and keyword counts	Entity extraction, skill coverage and similarity
Skill gaps	Not reported	Matched and missing skills per domain
Learning guidance	None	Course and resource recommendation
Explainability	Low	Score built from visible components
Domain advice	Rare	Ranking of all supported domains
Cost	Often paid	Open-source components

Advantages

- Gives each user a personalised, actionable report.
- Reduces dependence on informal feedback.
- Easy to extend with new domains, skills and courses.
- Fast response and low hardware requirements.

3.3 SYSTEM ARCHITECTURE

Architectural Overview

CareerBoost AI follows a layered, modular architecture (Figure 3.1). The **user layer** consists of students, graduates and job seekers. The **frontend** provides registration and login, resume upload, domain selection and the results dashboard. The **backend**, a Flask application, contains five processing modules connected in sequence: resume parser, text preprocessing, information extraction, the NLP and ML engine, and the recommendation engine. A **MySQL database** stores users, resumes, extracted information, skills, job roles, courses, recommendations and analytics logs. **External services** include the predefined job-role and skill database, course and learning-platform information and an optional LLM interface for enhanced insights. The **output layer** delivers the resume summary, skill-match score, skill-gap analysis, course recommendations and improvement suggestions.

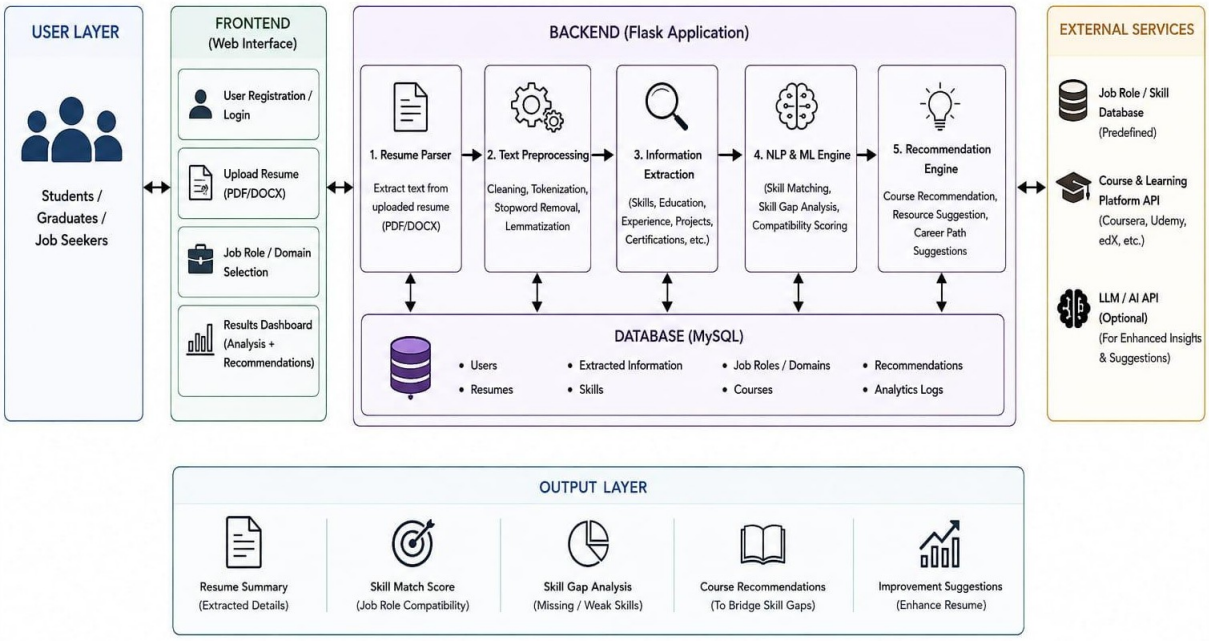


Figure 3.1: System architecture of CareerBoost AI

Processing Pipeline

Figure 3.2 shows the seven stages of processing. The stages are described below.

Step 1: Resume Upload. The user uploads a resume in PDF or DOCX format and the system stores the file securely for the duration of the analysis.

Step 2: Resume Parsing. Text is extracted from the file and divided into sections such as contact details, education, experience, skills and projects.

Step 3: Text Preprocessing. Noise is removed and the text is tokenised, stripped of stop-words and lemmatised.

Step 4: Information Extraction. Key entities such as skills, education, experience, projects, certifications and technologies are identified.

Step 5: Skill Matching and Gap Analysis. The extracted skills are matched with the selected domain, a compatibility score is calculated, missing skills are identified and strengths and weaknesses are classified.

Step 6: Recommendation Engine. Courses, books, websites and videos are recommended for the missing skills.

Step 7: Results and Dashboard. The analysis, score, skill gaps, recommendations and improvement tips are displayed. The user can update the resume, learn the skills and re-analyse.

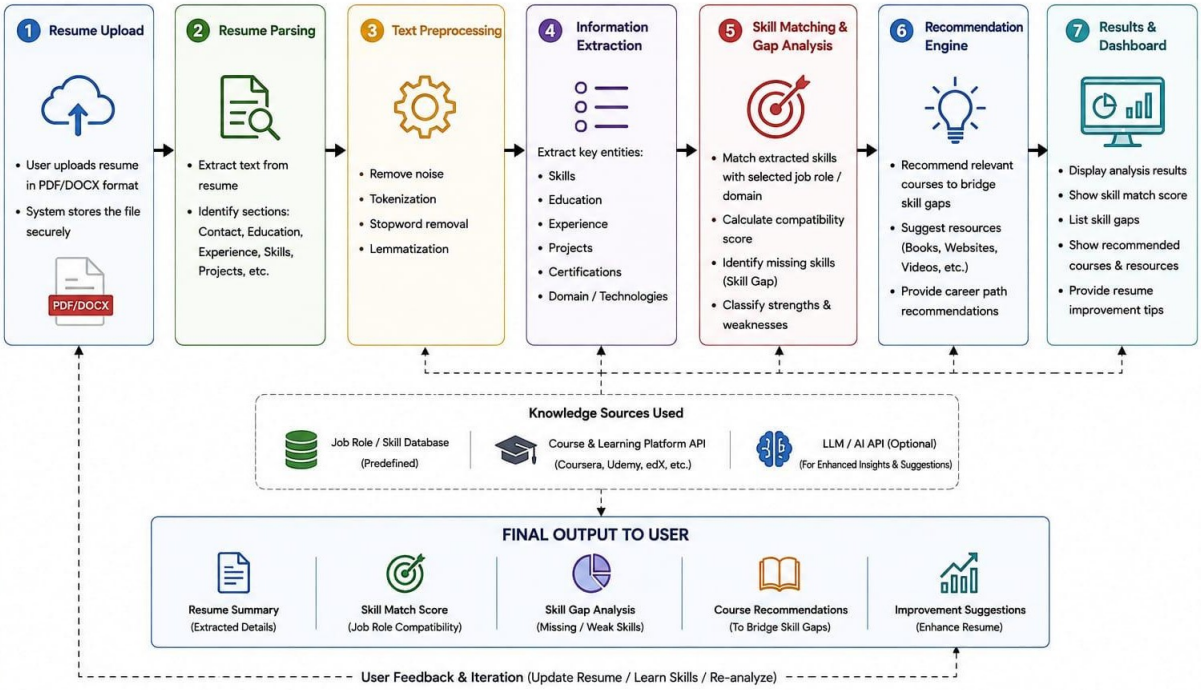


Figure 3.2: Processing pipeline of CareerBoost AI

Data Flow Diagram (DFD)

The context-level diagram (Figure 3.3) shows CareerBoost AI as a single process that exchanges data with the user and the administrator and uses three data stores. The level-1 diagram (Figure 3.4) decomposes the process into six sub-processes.

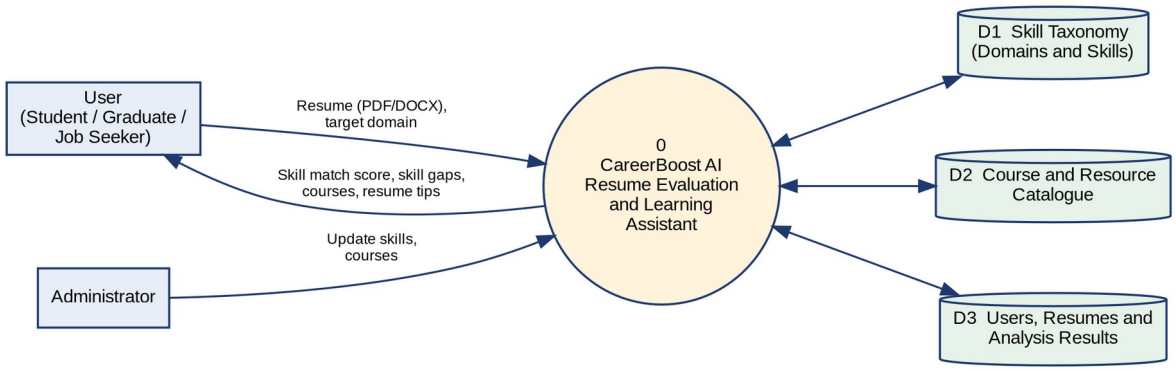


Figure 3.3: Data flow diagram, Level 0 (context diagram)

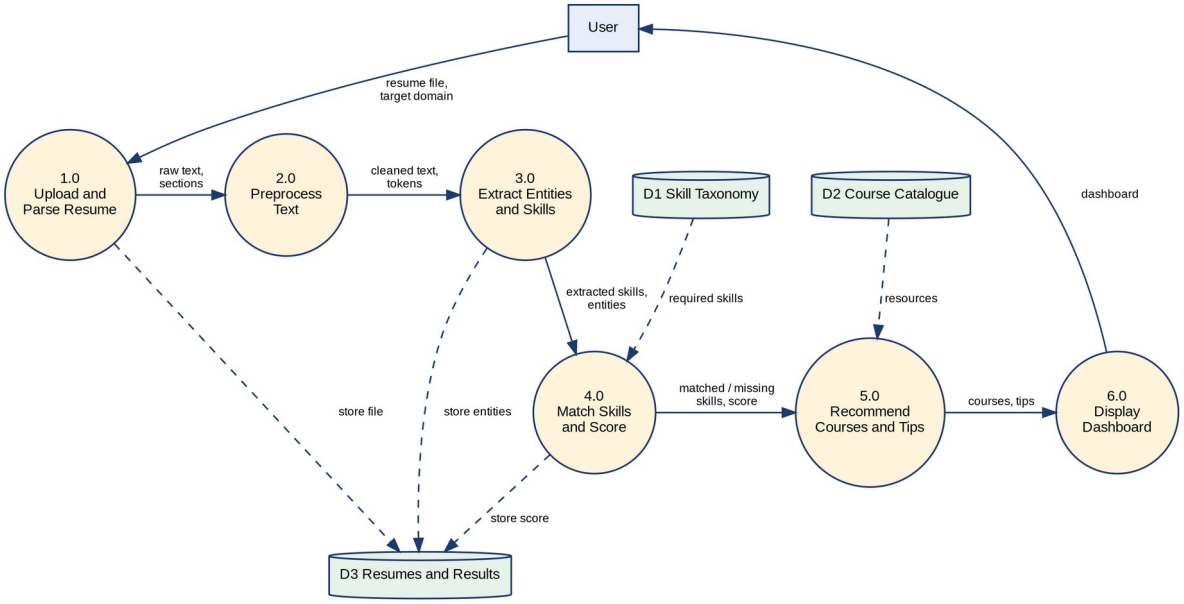


Figure 3.4: Data flow diagram, Level 1

Use Case Diagram

Two actors interact with the system (Figure 3.5). The user registers, uploads a resume, selects a domain and views the score, skill gaps, course recommendations and resume tips. The administrator maintains the skill taxonomy and the course catalogue.

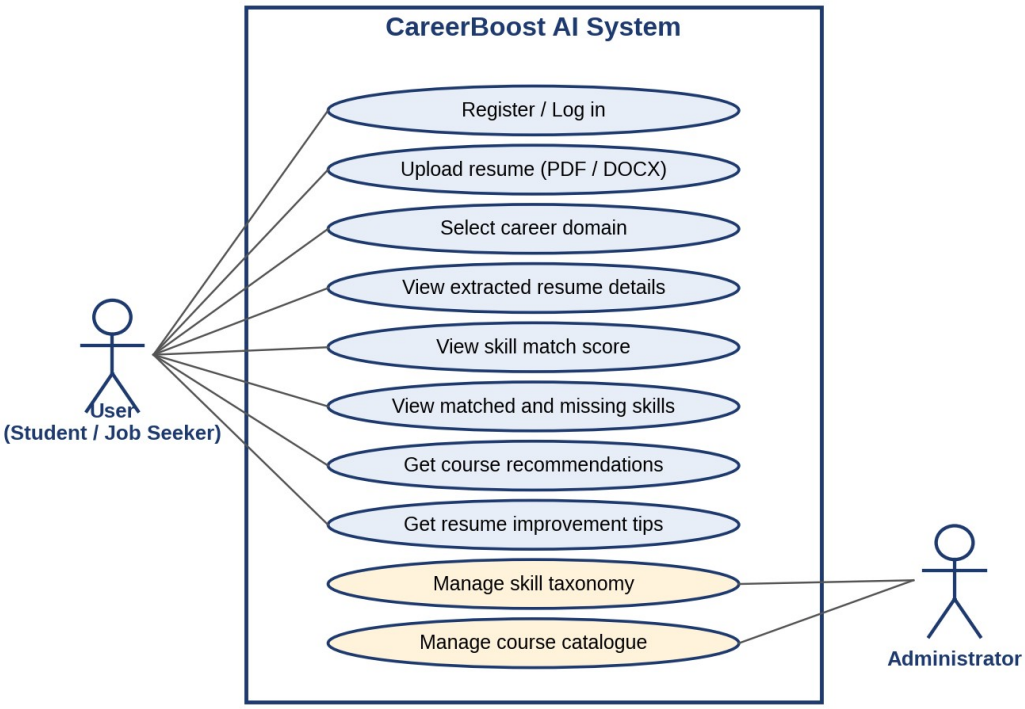


Figure 3.5: Use case diagram

System Flowchart

Figure 3.6 shows the decision logic of one analysis request, including the two validation checks that stop processing early when a file is unsupported or unreadable.

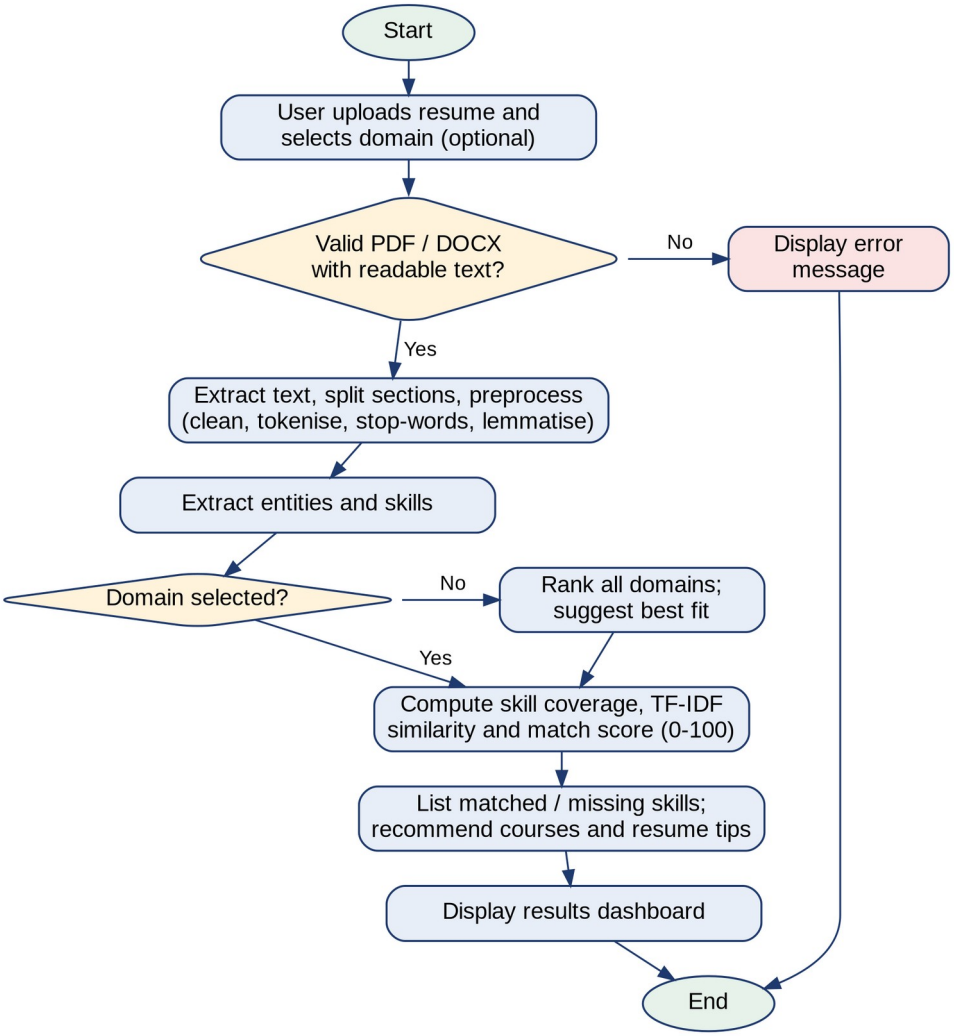


Figure 3.6: Flowchart of the resume analysis process

Sequence Diagram

Figure 3.7 shows the order of interactions between the user, the web interface, the Flask server, the NLP engine, the matching engine and the database during one request.

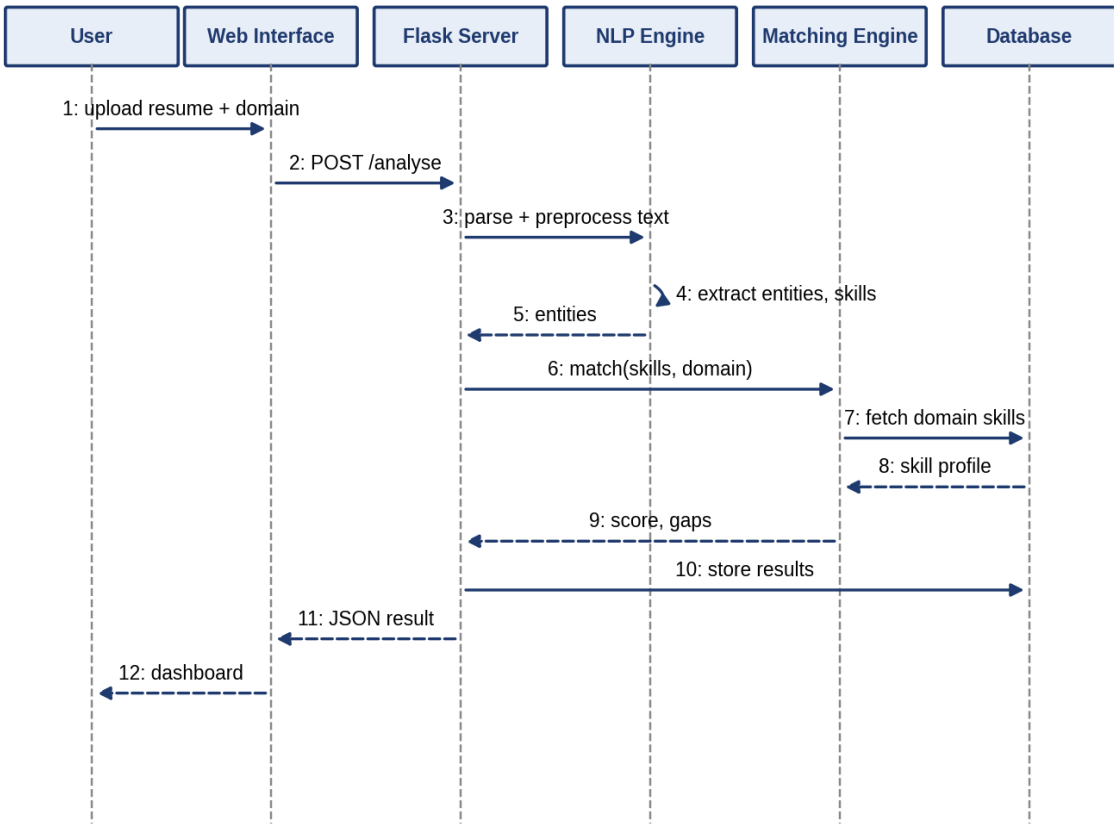


Figure 3.7: Sequence diagram of a resume analysis request

Database Design

The MySQL schema is shown as an entity-relationship diagram in Figure 3.8. A user uploads many resumes; each resume has one extracted-information record, many detected skills and many analyses. Domains and skills are linked through a weighted DOMAIN_SKILLS table, and each analysis can have many recommendations that point to catalogue courses.

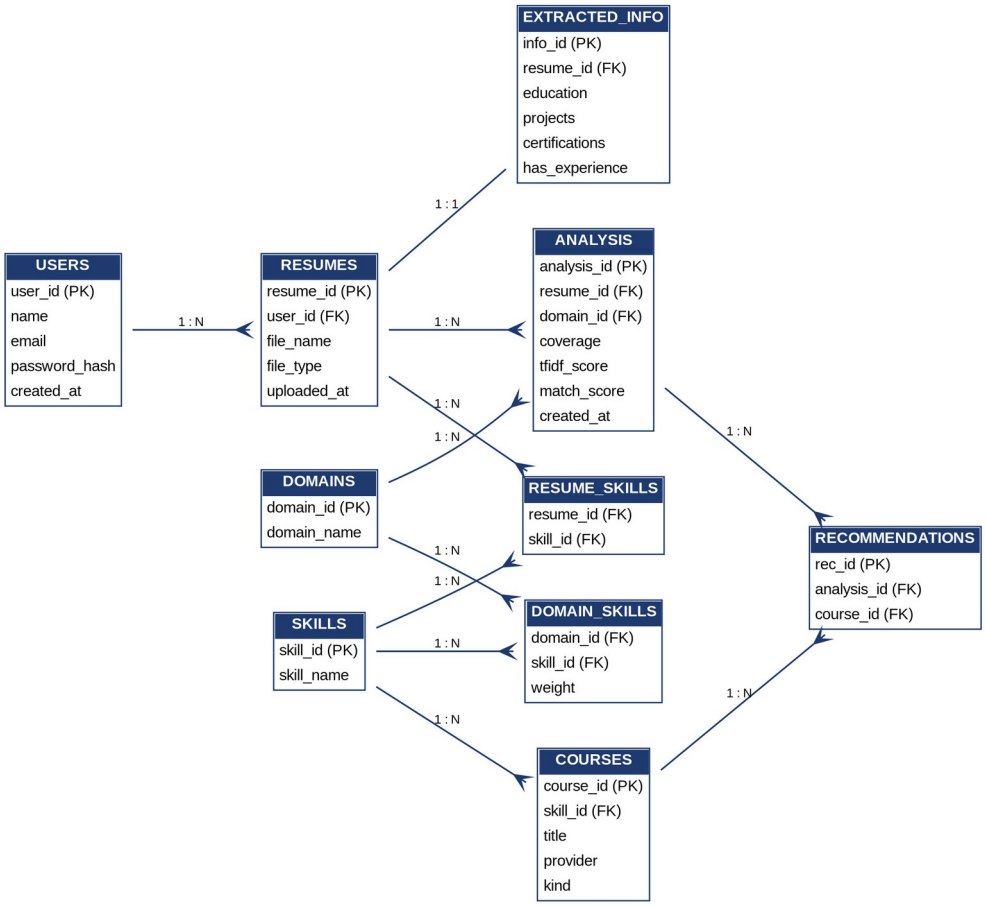


Figure 3.8: Entity-relationship diagram

3.4 HARDWARE REQUIREMENTS

Table 3.2: Hardware requirements

Component	Minimum requirement	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 or higher (or equivalent AMD)
RAM	4 GB	8 GB or higher
Storage	1 GB free space	2 GB or more (for models and datasets)
GPU	Not required	NVIDIA GPU, only for fine-tuning Sentence-BERT
Network	Required for installation and course links	Broadband internet connection
Display	1366 × 768	1920 × 1080

3.5 SOFTWARE REQUIREMENTS

Table 3.3: Software requirements

Category	Technology	Purpose
Programming language	Python 3.11 / 3.12	Core logic and ML
Frontend	HTML, CSS, JavaScript, Bootstrap	User interface

Category	Technology	Purpose
Backend framework	Flask	Web server and routing
NLP	NLTK, spaCy	Tokenisation, stop-words, lemmatisation, entity support
Machine learning	Scikit-learn (TF-IDF, cosine similarity)	Vectorisation and similarity
Semantic matching	Sentence-BERT (sentence-transformers)	Optional semantic similarity
Resume parsing	PyMuPDF, python-docx	PDF and DOCX text extraction
Database	MySQL	Persistent storage
Visualisation	Chart.js	Dashboard charts
Development tool	Visual Studio Code	Coding and debugging
Browser	Google Chrome or Mozilla Firefox	Accessing the dashboard
Operating system	Windows 10/11 or Ubuntu 22.04+	Execution environment

CHAPTER 4. IMPLEMENTATION

4.1 METHODOLOGY

The methodology describes the systematic approach followed to develop CareerBoost AI. It explains how a resume is converted into a structured profile, how that profile is compared with a career domain and how the score and recommendations are produced. The development was applied and experimental in nature: each module was built, tested on sample resumes and refined before the next module was added.

Research Approach

The approach combines rule-based extraction with machine-learning-based similarity so that the system is accurate on well-defined skills yet tolerant of wording differences. It consists of:

1. **Requirement Analysis:** Defining the problem as skill-gap analysis against a chosen career domain.
2. **Taxonomy Construction:** Building a weighted skill taxonomy for each supported domain.
3. **Pipeline Development:** Implementing parsing, preprocessing, extraction, matching and recommendation as separate modules.
4. **Evaluation:** Verifying the behaviour with functional test cases and response-time measurements on sample resumes.

System Development Methodology

The project follows a modular development methodology with the following sequential but interlinked stages:

1. Resume Upload and Parsing
2. Text Preprocessing
3. Information Extraction
4. Skill Matching, Gap Analysis and Scoring
5. Recommendation and Resume Tips
6. Dashboard Presentation

Skill Taxonomy Design

A skill taxonomy is the knowledge base that defines what a domain expects. Four domains were defined: Data Science, Web Development, Full Stack Development and Cybersecurity. Each domain lists **core skills** (essential for entry-level roles, weight 2) and **optional skills** (valuable but not essential, weight 1). A synonym table maps alternative spellings to one canonical skill; for example "sklearn" and "scikit learn" map to "scikit-learn", and "nodejs" maps to "node.js". Table 4.1 summarises the taxonomy.

Table 4.1: Skill taxonomy used by the prototype

Domain	Core skills (weight 2)	Optional skills (weight 1)	Total weight
Data Science	python, sql, statistics, pandas, numpy, data visualization, machine learning, scikit-learn, data cleaning (9)	deep learning, tensorflow, pytorch, power bi, tableau, nlp, feature engineering, big data (8)	26
Web Development	html, css, javascript, responsive design, git, bootstrap, react, rest api (8)	typescript, tailwind css, seo, figma, web accessibility, jquery (6)	22
Full Stack Development	html, css, javascript, react, node.js, express, sql, mongodb, rest api, git (10)	docker, typescript, aws, ci/cd, flask, django, graphql, jwt (8)	28
Cybersecurity	networking, linux, cryptography, network security, vulnerability assessment, penetration testing, owasp, incident response, siem (9)	wireshark, nmap, metasploit, burp suite, python, ethical hacking, security+, risk assessment (8)	26

Text Preprocessing

Raw resume text contains noise such as e-mail addresses, links, symbols and inconsistent capitalisation. The preprocessing stage applies the following steps in order:

1. **Cleaning:** Conversion to lower case; removal of links, e-mail addresses and special characters while keeping characters that occur in skill names (+, #, . and /).
2. **Tokenisation:** Splitting the text into word tokens (NLTK) [16].
3. **Stop-word removal:** Deleting common English words that carry no skill information.
4. **Lemmatisation:** Reducing words to their base form using the WordNet lemmatiser.

The cleaned string is used for skill matching and similarity, while the token list is used for text statistics.

Feature Representation: TF-IDF

To compare a resume with a domain, both are converted into numerical vectors. In the TF-IDF scheme the weight of a term *t* in a document *d* is the product of its frequency in the document and its inverse document frequency [13], [14]:

$$tf\text{-}idf(t, d) = tf(t, d) \times idf(t)$$

In Scikit-learn [18] the default smoothed inverse document frequency is idf(t) = ln((1 + N) / (1 + df(t))) + 1, where N is the number of documents and df(t) the number of documents containing t. The prototype builds the vectors from one-word and two-word terms, so phrases such as "machine learning" are represented. Each domain is represented by a **profile document** made of its skills, with core skills repeated twice to reflect their higher weight.

Cosine Similarity

The similarity between the resume vector r and a domain-profile vector p is the cosine of the angle between them [15]:

$$cos(r, p) = (r \cdot p) / (\|r\| \times \|p\|)$$

The value lies between 0 (no shared terms) and 1 (identical direction). Because TF-IDF vectors are non-negative, the result is never negative.

Optional Semantic Layer: Sentence-BERT

Keyword vectors cannot recognise that "neural networks" and "deep learning" are related. Sentence-BERT encodes each text into a dense embedding so that semantically related texts have a high cosine similarity [9]. The prototype includes a function that encodes the resume and the domain profile with a compact pre-trained model (all-MiniLM-L6-v2, a MiniLM-based model [12]) and returns their cosine similarity. This layer is optional: it requires the sentence-transformers library and the downloaded model weights, and the results reported in Chapter 5 were obtained with the TF-IDF and skill-coverage components.

Scoring Model

The skill-match score for a resume R and a domain D combines two measures (Figure 4.1). **Weighted skill coverage** C is the share of the domain's total skill weight that is present in the resume. **TF-IDF cosine similarity** T measures overall textual closeness to the domain profile. The final score on a 0 to 100 scale is:

$$Score = 100 \times (0.7\ C + 0.3\ T)$$

The weights 0.7 and 0.3 are design parameters that can be tuned. Coverage is given the larger weight because it is directly interpretable: it states how much of the domain's required skill set has been demonstrated. When an optional semantic term B is available, the score generalises to 100 × (a C + b T + c B) with a + b + c = 1.

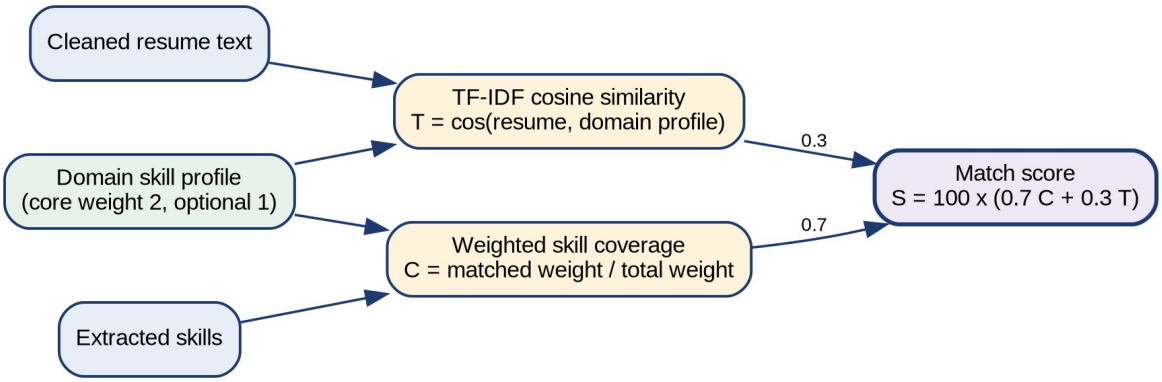


Figure 4.1: Skill-match scoring model

Worked example. For Resume A (Chapter 5) and the Data Science domain, all nine core skills are present (weight 18) and none of the eight optional skills are present, so C = 18 / 26 = 0.692. The TF-IDF cosine similarity is T = 0.213. The score is 100 × (0.7 × 0.692 + 0.3 × 0.213) = 54.9.

Domain Suggestion and Classification Roadmap

When the user does not choose a domain, the prototype calculates the score for every domain and ranks them; the highest-scoring domain is suggested. This profile-similarity ranking acts as an unsupervised baseline for the career-domain prediction described in the project abstract. A supervised classifier, such as logistic regression, a support vector machine or a random forest trained on TF-IDF features [18], can replace the baseline once a labelled resume dataset has been collected.

Recommendation Logic

The recommender takes the missing skills in priority order, core skills first and then optional skills, and looks up each skill in a resource catalogue. Each entry gives a title, a provider and a type (course, free tutorial, documentation, certification or practice path). One primary resource is returned per skill and the list is capped at six items to keep the dashboard readable. A skill without a catalogue entry receives a generic suggestion to search learning platforms. In addition, simple rules generate resume-improvement tips: add a summary, add a projects section, list certifications, include GitHub and LinkedIn links, quantify project results and keep the length reasonable.

Evaluation Methodology

Because no labelled benchmark was available, the prototype was evaluated in two ways. First, thirteen functional test cases check parsing, validation, scoring, recommendation and web-endpoint behaviour. Second, response time was measured over repeated runs. Three sample resumes, one for each of three domains, were written for the demonstration. For future evaluation with labelled data, the standard measures of precision, recall and F1-score for skill extraction and domain prediction should be used [20].

4.2 IMPLEMENTATION

The prototype is implemented in Python as six small modules plus a Flask application. The complete source code is given in Section 5.3.

4.2.1 Resume Upload and Parsing Module

This module (parser.py) accepts a file path and returns the raw text. PDF files are read page by page with PyMuPDF; DOCX files are read paragraph by paragraph, including text in tables, with python-docx. Files with any other extension raise a parsing error, as do corrupt files and files from which fewer than thirty characters can be read. The function split_sections() then

walks through the lines of the text and groups them under recognised headings: summary, education, skills, projects, experience, certifications and achievements. Heading recognition uses regular expressions that also accept common variants such as "Technical Skills", "Internships" and "Courses".

4.2.2 Text Preprocessing Module

The module (preprocess.py) implements the four preprocessing steps described in Section 4.1. The clean_text() function lower-cases the text, removes links and bare web addresses, removes e-mail addresses and discards characters that are not letters, digits or the symbols + # . / and -. The preprocess() function then tokenises the cleaned text with NLTK, removes English stop-words and applies the WordNet lemmatiser. It returns both the cleaned string and the token list.

4.2.3 Information Extraction Module

The module (extractor.py) builds the structured profile of the candidate. The name is taken from the first non-empty line; the e-mail address and a ten-digit phone number are found with regular expressions; degrees such as B.Tech, B.E., M.Tech and MCA are detected inside the education section. Skills are found by matching every taxonomy skill and its synonyms in the cleaned text with boundary-aware patterns, which prevents, for example, "sql" from matching inside "nosql". The output includes the skill list, project lines, certification lines, an experience flag and the list of sections found.

4.2.4 Skill Matching and Gap Analysis Module

The module (matcher.py) implements the scoring model. skill_gap() separates the domain's skills into matched skills, missing core skills and missing optional skills and calculates the weighted coverage. tfidf_similarity() fits a TF-IDF vectoriser on the four domain profiles together with the resume and returns the cosine similarity to each profile. match_score() combines the two values into the final score, and rank_domains() repeats the calculation for every domain to suggest the best fit. The optional function sbert_similarity() provides the semantic layer.

4.2.5 Recommendation and Dashboard Module

The module (recommender.py) contains the resource catalogue, the recommend() function that maps missing skills to resources and the resume_tips() function that applies the improvement rules. The pipeline (pipeline.py) connects all modules in the order upload, parse, preprocess, extract, rank, match, recommend and returns one result object. The Flask application (app.py) exposes this pipeline through an upload form and an /analyse endpoint that returns the result as JSON. The designed Chart.js dashboard (Chapter 3) consumes this JSON to draw the score card, the domain ranking, the skill lists and the recommendation list. In the present prototype the JSON is the delivered output, and the dashboard layouts shown in Chapter 5 were generated from it with Matplotlib to illustrate the intended screen.

4.3 DATABASE DESIGN AND IMPLEMENTATION STATUS

The persistent schema (Figure 3.8) contains ten tables: USERS, RESUMES, EXTRACTED_INFO, ANALYSIS, DOMAINS, SKILLS, DOMAIN_SKILLS, RESUME_SKILLS, COURSES and RECOMMENDATIONS. In the current prototype the taxonomy and the resource catalogue are held in Python data structures and each request is processed statelessly. Moving these structures into the MySQL tables, together with user accounts and analysis history, is the next development increment and does not change the processing modules.

4.4 SECURITY AND PRIVACY CONSIDERATIONS

Resumes contain personal data, so the prototype applies the following safeguards:

- **File validation:** Only .pdf and .docx extensions are accepted, and the upload size is limited to 5 MB.
- **Temporary storage:** The uploaded file is saved to a temporary location and deleted as soon as processing ends, whether it succeeds or fails.
- **Error handling:** Corrupt, empty or unsupported files return a clear error message (HTTP 400 or 422) and never crash the server.
- **Planned measures:** Password hashing for user accounts, HTTPS in deployment and a user option to delete stored results.

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

The project titled "CareerBoost AI: Resume Evaluation and Learning Assistant Using Machine Learning" was undertaken to address a practical problem faced by students and fresh graduates: they cannot easily tell whether their resume reflects the skills expected in a chosen career, and existing tools give little guidance on what to learn. This project provides a web-based, explainable system that reads a PDF or DOCX resume, extracts its key entities, compares its skills with a career domain and recommends learning resources for the gaps.

The design combines rule-based extraction with machine-learning-based similarity. A weighted skill taxonomy and a synonym table give precise and interpretable skill matching; TF-IDF vectors and cosine similarity add a measure of overall closeness to the domain; and Sentence-BERT is included as an optional semantic layer for wording differences. The final score is a transparent combination of weighted skill coverage and similarity, so the user can see exactly why a score was given.

Purpose and Achievements. The main achievements of the project are:

- A working parser for PDF and DOCX resumes with validation of unsupported, corrupt and empty files.
- An NLP preprocessing pipeline of cleaning, tokenisation, stop-word removal and lemmatisation.
- Synonym-aware skill extraction against a four-domain weighted taxonomy.
- A skill-match score with matched and missing skills, and a ranking of all domains.
- A recommendation module that links every skill in the taxonomy to a learning resource and generates resume-improvement tips.
- A Flask web endpoint and a suite of 13 functional tests, of which 13 passed.

Significance. The system is aimed at candidates, not recruiters. It can be used by students for self-assessment, by placement cells for consistent batch feedback and by training institutes to find common skill gaps. Because it is built entirely with open-source libraries, it can be adopted at no software cost.

Limitations observed. The evaluation in this report used three illustrative resumes and no labelled benchmark; the results therefore demonstrate correct behaviour of the pipeline and are not a statistical measure of accuracy. Skill coverage is limited to the taxonomy, and closely related domains such as Web Development and Full Stack Development can receive similar scores.

5.2 FUTURE WORK

The following enhancements are proposed to extend the system:

- **Labelled dataset and supervised classifier:** Collect and label a resume dataset to train and evaluate a domain classifier, and report precision, recall and F1-score [20].
- **Fine-tuned semantic matching:** Fine-tune a Sentence-BERT model on resume and job-description pairs to improve matching beyond keyword overlap [2], [9].
- **Live job-market data:** Update domain skill requirements automatically from current job postings.
- **Multilingual resume support:** Parse and analyse resumes written in languages other than English.
- **OCR for scanned resumes:** Add optical character recognition for image-only PDFs.
- **Job-portal compatibility:** Integrate with major job portals so that results can be used directly in applications.
- **Persistent user accounts and history:** Complete the MySQL implementation so that users can track improvement across resume versions.
- **Larger taxonomy:** Add more domains, such as cloud computing, DevOps, mobile development and AI engineering.
- **LLM-assisted insights:** Use a large language model to rewrite resume sentences and generate personalised learning roadmaps.
- **Fairness checks:** Test the scoring for bias across institutions, genders and resume styles before wide deployment.

5.3 SOURCE CODE

The prototype consists of the Python files listed below. The taxonomy and resource catalogue are abridged in the listing for space; all other code is reproduced in full. The code was executed with Python 3.12 using Flask, scikit-learn, NLTK, PyMuPDF and python-docx.

Listing 5.1: skills_db.py

```
"""skills_db.py - Domain skill taxonomy used by CareerBoost AI.
Each domain lists CORE skills (weight 2) and OPTIONAL skills (weight 1).
SYNONYMS maps alternative spellings to the canonical skill name."""

DOMAINS = {
    "Data Science": {
        "core": ["python", "sql", "statistics", "pandas", "numpy",
                 "data visualization", "machine learning", "scikit-learn",
                 "data cleaning"],
        "optional": ["deep learning", "tensorflow", "pytorch", "power bi",
                     "tableau", "nlp", "feature engineering", "big data"],
    },
    "Web Development": {
        "core": ["html", "css", "javascript", "responsive design", "git",
```

```
                "bootstrap", "react", "rest api"],
        "optional": ["typescript", "tailwind css", "seo", "figma",
                     "web accessibility", "jquery"],
    },
    "Full Stack Development": {
        "core": ["html", "css", "javascript", "react", "node.js", "express",
                 "sql", "mongodb", "rest api", "git"],
        "optional": ["docker", "typescript", "aws", "ci/cd", "flask",
                     "django", "graphql", "jwt"],
    },
    "Cybersecurity": {
        "core": ["networking", "linux", "cryptography", "network security",
                 "vulnerability assessment", "penetration testing", "owasp",
                 "incident response", "siem"],
        "optional": ["wireshark", "nmap", "metasploit", "burp suite",
                     "python", "ethical hacking", "security+",
                     "risk assessment"],
    },
}

SYNONYMS = {
    "scikit-learn": ["sklearn", "scikit learn"],
    "machine learning": ["ml"],
    "nlp": ["natural language processing"],
    "deep learning": ["neural networks", "cnn", "rnn"],
    "data visualization": ["data visualisation", "matplotlib", "seaborn"],
    "data cleaning": ["data preprocessing", "data wrangling"],
    "power bi": ["powerbi"],
    "javascript": ["js", "es6"],
    "node.js": ["nodejs", "node js"],
    "express": ["express.js", "expressjs"],
    "react": ["react.js", "reactjs"],
    "rest api": ["restful api", "rest apis", "restful apis", "api development"],
    "mongodb": ["mongo db", "mongo"],
    "sql": ["mysql", "postgresql", "sqlite"],
    "css": ["css3"],
    "html": ["html5"],
    "responsive design": ["responsive web design", "mobile-first"],
    "networking": ["tcp/ip", "computer networks", "dns"],
    "network security": ["firewall", "firewalls", "ids/ips"],
    "penetration testing": ["pentesting", "pen testing", "pentest"],
    "vulnerability assessment": ["vulnerability scanning", "nessus"],
    "ci/cd": ["github actions", "jenkins", "continuous integration"],
    "ethical hacking": ["ctf", "capture the flag"],
    "risk assessment": ["risk management"],
    "cryptography": ["encryption", "aes", "rsa"],
    "siem": ["splunk", "qradar"],
    "jwt": ["json web token", "json web tokens"],
    "git": ["github", "gitlab", "version control"],
    "tailwind css": ["tailwind"],
    "web accessibility": ["wcag", "accessibility"],
}
```

Listing 5.2: parser.py

```
"""parser.py - Extract raw text and section blocks from PDF / DOCX resumes."""
import os
import re
import fitz                     # PyMuPDF
from docx import Document       # python-docx
```

```
ALLOWED = {".pdf", ".docx"}
SECTION_HEADINGS = {
    "summary": r"(professional )?(summary|objective|profile|about me)",
    "education": r"education(al)?( qualifications?| background)?",
    "skills": r"(technical )?skills?( summary)?|core competencies",
    "projects": r"(academic |personal )?projects?",
    "experience": r"(work )?experience|internships?|employment",
    "certifications": r"certifications?|courses?( completed)?|licen[cs]es?",
    "achievements": r"achievements?|awards?|honou?rs",
}

class ResumeParseError(Exception):
    pass

def extract_text(path):
    ext = os.path.splitext(path)[1].lower()
    if ext not in ALLOWED:
        raise ResumeParseError("Unsupported file type: " + ext)
    try:
        if ext == ".pdf":
            with fitz.open(path) as pdf:
                text = "\n".join(page.get_text() for page in pdf)
        else:
            doc = Document(path)
            parts = [p.text for p in doc.paragraphs]
            for table in doc.tables:
                for row in table.rows:
                    parts.extend(cell.text for cell in row.cells)
            text = "\n".join(parts)
    except Exception as exc:
        raise ResumeParseError("Corrupt or unreadable file: %s" % exc)
    if len(text.strip()) < 30:
        raise ResumeParseError("No readable text found in the resume")
    return text

def split_sections(text):
    """Group resume lines under the most recent recognised heading."""
    sections, current = {"header": []}, "header"
    for line in text.splitlines():
        stripped = line.strip().strip(":").strip()
        if not stripped:
            continue
        matched = None
        if len(stripped) <= 40:
            for name, pattern in SECTION_HEADINGS.items():
                if re.fullmatch(pattern, stripped, flags=re.I):
                    matched = name
                    break
        if matched:
            current = matched
            sections.setdefault(current, [])
        else:
            sections.setdefault(current, []).append(stripped)
    return {k: "\n".join(v) for k, v in sections.items() if v}
```

Listing 5.3: preprocess.py

```
"""preprocess.py - Cleaning, tokenisation, stop-word removal, lemmatisation."""
import re
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
```

```
from nltk.tokenize import word_tokenize

_STOP = set(stopwords.words("english"))
_LEM = WordNetLemmatizer()

def clean_text(text):
    text = text.lower()
    text = re.sub(r"https?://\S+|www\.\S+", " ", text)
    text = re.sub(r"\b[\w-]+\.(com|in|org|io|net)/\S*", " ", text)
    text = re.sub(r"\S+@\S+", " ", text)
    text = re.sub(r"[^a-z0-9+#./\s-]", " ", text)
    return re.sub(r"\s+", " ", text).strip()

def preprocess(text):
    """Return the cleaned string and its lemmatised, stop-word-free tokens."""
    cleaned = clean_text(text)
    tokens = [t for t in word_tokenize(cleaned)
              if t not in _STOP and len(t) > 1]
    tokens = [_LEM.lemmatize(t) for t in tokens]
    return cleaned, tokens
```

Listing 5.4: extractor.py

```
"""extractor.py - Entity extraction: contact data, education and skills."""
import re
from skills_db import DOMAINS, SYNONYMS

EMAIL = re.compile(r"[\w.+-]+@[\w-]+\.[\w.-]+")
PHONE = re.compile(r"(?:\+?\d{1,3}[\s-]?)?\d{10}")
DEGREE = re.compile(r"\b(b\.?\s?tech|b\.?\s?e\b|b\.?\s?sc|b\.?\s?c\.?a|"
                    r"m\.?\s?tech|m\.?\s?e\b|m\.?\s?sc|m\.?\s?c\.?a|mba|"
                    r"diploma)\b", re.I)

ALL_SKILLS = sorted({s for d in DOMAINS.values()
                     for k in ("core", "optional") for s in d[k]})

def _variants(skill):
    return [skill] + SYNONYMS.get(skill, [])

def _pattern(term):
    return r"(?<![a-z0-9+#])" + re.escape(term) + r"(?![a-z0-9+#])"

def extract_skills(cleaned_text):
    """Return the set of canonical skills found in the cleaned resume text."""
    found = set()
    for skill in ALL_SKILLS:
        if any(re.search(_pattern(v), cleaned_text) for v in _variants(skill)):
            found.add(skill)
    return found

def extract_entities(raw_text, sections, cleaned_text):
    lines = [l.strip() for l in raw_text.splitlines() if l.strip()]
    email = EMAIL.search(raw_text)
    phone = PHONE.search(raw_text.replace(" ", ""))
    degrees = sorted({m.group(0).upper().replace(" ", "")
                      for m in DEGREE.finditer(sections.get("education", ""))})
    return {
        "name": lines[0] if lines else "Unknown",
        "email": email.group(0) if email else None,
        "phone": phone.group(0) if phone else None,
        "degrees": degrees,
```

```
        "skills": sorted(extract_skills(cleaned_text)),
        "projects": [l for l in sections.get("projects", "").split("\n") if l],
        "certifications": [l for l in
                           sections.get("certifications", "").split("\n") if l],
        "has_experience": "experience" in sections,
        "sections_found": sorted(k for k in sections if k != "header"),
    }
```

Listing 5.5: matcher.py

```
"""matcher.py - Skill-gap analysis, TF-IDF similarity and match scoring.
Final score = ALPHA * weighted skill coverage + BETA * TF-IDF cosine."""
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from skills_db import DOMAINS

ALPHA, BETA = 0.7, 0.3          # tunable scoring weights
CORE_W, OPT_W = 2, 1            # skill importance weights

def domain_profile(domain):
    spec = DOMAINS[domain]
    return " ".join([s for s in spec["core"] for _ in range(CORE_W)] +
                    [s for s in spec["optional"] for _ in range(OPT_W)])

def skill_gap(skills, domain):
    spec, have = DOMAINS[domain], set(skills)
    matched = [s for s in spec["core"] + spec["optional"] if s in have]
    missing_core = [s for s in spec["core"] if s not in have]
    missing_opt = [s for s in spec["optional"] if s not in have]
    total = CORE_W * len(spec["core"]) + OPT_W * len(spec["optional"])
    got = (CORE_W * sum(s in have for s in spec["core"]) +
           OPT_W * sum(s in have for s in spec["optional"]))
    return {"matched": matched, "missing_core": missing_core,
            "missing_optional": missing_opt, "coverage": got / total}

def tfidf_similarity(cleaned_resume):
    """Cosine similarity between the resume and every domain profile."""
    names = list(DOMAINS)
    docs = [domain_profile(n) for n in names] + [cleaned_resume]
    vec = TfidfVectorizer(token_pattern=r"[a-z0-9+#./-]+", ngram_range=(1, 2))
    matrix = vec.fit_transform(docs)
    sims = cosine_similarity(matrix[-1], matrix[:-1])[0]
    return dict(zip(names, (float(s) for s in sims)))

def match_score(skills, cleaned_resume, domain):
    gap = skill_gap(skills, domain)
    cos = tfidf_similarity(cleaned_resume)[domain]
    gap["tfidf"] = cos
    gap["score"] = round(100 * (ALPHA * gap["coverage"] + BETA * cos), 1)
    return gap

def rank_domains(skills, cleaned_resume):
    """Suggest the best-fitting domain (used when the user has no target)."""
    scores = {d: match_score(skills, cleaned_resume, d)["score"]
              for d in DOMAINS}
    return dict(sorted(scores.items(), key=lambda kv: -kv[1]))

def sbert_similarity(resume_text, domain, model_name="all-MiniLM-L6-v2"):
    """Optional semantic layer (needs sentence-transformers + model weights)."""
    from sentence_transformers import SentenceTransformer, util
    model = SentenceTransformer(model_name)
```

```
    emb = model.encode([resume_text, domain_profile(domain)])
    return float(util.cos_sim(emb[0], emb[1]))
```

Listing 5.6: recommender.py

```
"""recommender.py - Learning resources for missing skills + resume tips."""

# skill -> list of (title, provider, kind)
RESOURCES = {
 "python": [("Python for Everybody Specialization", "Coursera (Univ. of Michigan)", "Course")],
 "sql": [("Intro to SQL", "Kaggle Learn", "Free course"), ("SQLBolt", "sqlbolt.com", "Free
tutorial")],
 "statistics": [("Statistics and Probability", "Khan Academy", "Free course")],
 "pandas": [("Pandas", "Kaggle Learn", "Free course")],
 "numpy": [("NumPy: the absolute basics for beginners", "NumPy documentation", "Free tutorial")],
 "data visualization": [("Data Visualization", "Kaggle Learn", "Free course")],
 "machine learning": [("Machine Learning Specialization", "Coursera (DeepLearning.AI / Stanford)",
"Course")],
 "scikit-learn": [("scikit-learn Tutorials", "scikit-learn documentation", "Free tutorial")],
 "data cleaning": [("Data Cleaning", "Kaggle Learn", "Free course")],
 "deep learning": [("Practical Deep Learning for Coders", "fast.ai", "Free course"), ("Deep Learning
Specialization", "Coursera (DeepLearning.AI)", "Course")],
 # ... remaining catalogue entries follow the same (title, provider, kind) format ...
}
FALLBACK = [("Search the skill name on Coursera, edX or freeCodeCamp", "Learning platforms",
"Search")]

def recommend(missing_core, missing_optional, limit=6):
    """Core gaps first, then optional gaps; capped to keep output readable."""
    out = []
    for skill in missing_core + missing_optional:
        for title, provider, kind in RESOURCES.get(skill, FALLBACK):
            out.append({"skill": skill, "title": title,
                        "provider": provider, "kind": kind})
            break                       # one primary resource per skill
        if len(out) == limit:
            break
    return out

def resume_tips(entities, raw_text):
    tips, words = [], len(raw_text.split())
    secs = set(entities["sections_found"])
    if "summary" not in secs:
        tips.append("Add a 2-3 line professional summary that names your target role.")
    if "projects" not in secs:
        tips.append("Add a Projects section describing what you built and the tools used.")
    if "certifications" not in secs:
        tips.append("List relevant certifications or completed online courses.")
    if "github" not in raw_text.lower() and "linkedin" not in raw_text.lower():
        tips.append("Include GitHub and LinkedIn profile links.")
    if not any(ch.isdigit() for ch in "".join(entities["projects"])):
        tips.append("Quantify project results (accuracy, users, speed-up, dataset size).")
    if words < 150:
        tips.append("The resume is very short (%d words); add detail on skills and projects." %
words)
    if words > 900:
        tips.append("The resume is long (%d words); trim it to one or two pages." % words)
    return tips
```

Listing 5.7: pipeline.py

```
"""pipeline.py - End-to-end analysis: upload -> parse -> NLP -> match -> advise."""
```

```
import time
from parser import extract_text, split_sections
from preprocess import preprocess
from extractor import extract_entities
from matcher import match_score, rank_domains
from recommender import recommend, resume_tips
from skills_db import DOMAINS

def analyse(path, target_domain=None):
    t0 = time.perf_counter()
    raw = extract_text(path)
    sections = split_sections(raw)
    cleaned, tokens = preprocess(raw)
    entities = extract_entities(raw, sections, cleaned)
    ranking = rank_domains(entities["skills"], cleaned)
    domain = target_domain or next(iter(ranking))
    if domain not in DOMAINS:
        raise ValueError("Unknown domain: " + domain)
    result = match_score(entities["skills"], cleaned, domain)
    return {
        "entities": entities,
        "token_count": len(tokens),
        "domain_ranking": ranking,
        "suggested_domain": next(iter(ranking)),
        "target_domain": domain,
        "match": result,
        "courses": recommend(result["missing_core"], result["missing_optional"]),
        "tips": resume_tips(entities, raw),
        "elapsed_ms": round((time.perf_counter() - t0) * 1000, 1),
    }
```

Listing 5.8: app.py

```
"""app.py - Flask web interface for CareerBoost AI."""
import os
import tempfile
from flask import Flask, request, jsonify, render_template_string
from pipeline import analyse
from parser import ResumeParseError, ALLOWED
from skills_db import DOMAINS

app = Flask(__name__)
app.config["MAX_CONTENT_LENGTH"] = 5 * 1024 * 1024      # 5 MB upload limit

PAGE = """<!doctype html><title>CareerBoost AI</title>
<h2>CareerBoost AI - Resume Evaluation</h2>
<form method=post action=/analyse enctype=multipart/form-data>
 <input type=file name=resume accept=.pdf,.docx required>
 <select name=domain><option value="">Auto-detect domain</option>
 {% for d in domains %}<option>{{d}}</option>{% endfor %}</select>
 <button>Analyse</button></form>"""

@app.get("/")
def index():
    return render_template_string(PAGE, domains=list(DOMAINS))

@app.post("/analyse")
def analyse_route():
    file = request.files.get("resume")
    if file is None or file.filename == "":
        return jsonify(error="No file uploaded"), 400
```

```
    ext = os.path.splitext(file.filename)[1].lower()
    if ext not in ALLOWED:
        return jsonify(error="Only PDF and DOCX files are accepted"), 400
    with tempfile.NamedTemporaryFile(suffix=ext, delete=False) as tmp:
        file.save(tmp.name)
    try:
        return jsonify(analyse(tmp.name, request.form.get("domain") or None))
    except ResumeParseError as exc:
        return jsonify(error=str(exc)), 422
    except ValueError as exc:
        return jsonify(error=str(exc)), 400
    finally:
        os.remove(tmp.name)

if __name__ == "__main__":
    app.run(debug=False)
```

Listing 5.9: test_pipeline.py

```
"""test_pipeline.py - Functional tests for CareerBoost AI (run: python test_pipeline.py)."""
import io, os, sys, json, time, tempfile, statistics, warnings
warnings.filterwarnings("ignore")
from pipeline import analyse
from parser import ResumeParseError
from skills_db import DOMAINS
from recommender import RESOURCES
from app import app

S = "../samples/"
A, B, C = S + "resume_A_ananya.docx", S + "resume_B_karthik.pdf", S + "resume_C_meera.docx"
results = []

def case(tid, desc, fn):
    try:
        fn(); results.append((tid, desc, "Pass"))
    except AssertionError as e:
        results.append((tid, desc, "FAIL: %s" % e))

def t1(): r = analyse(B); assert r["entities"]["email"] == "karthik.s@example.com"
def t2(): r = analyse(A); assert r["entities"]["name"] == "Ananya Rao"
def t3():
    with tempfile.NamedTemporaryFile(suffix=".txt", delete=False) as f: f.write(b"hello" * 20)
    try: analyse(f.name); assert False, "accepted .txt"
    except ResumeParseError: pass
def t4():
    with tempfile.NamedTemporaryFile(suffix=".pdf", delete=False) as f: f.write(b"not a real pdf")
    try: analyse(f.name); assert False, "accepted corrupt file"
    except ResumeParseError: pass
def t5():
    from docx import Document
    d = Document(); d.save("/tmp/empty.docx")
    try: analyse("/tmp/empty.docx"); assert False, "accepted empty docx"
    except ResumeParseError: pass
def t6():
    for d in DOMAINS:
        r = analyse(A, d); assert r["target_domain"] == d
def t7():
    for f in (A, B, C):
        for d in DOMAINS:
            assert 0 <= analyse(f, d)["match"]["score"] <= 100
def t8():
```

```
    for f in (A, B, C):
        r = analyse(f)
        n = len(r["match"]["missing_core"]) + len(r["match"]["missing_optional"])
        assert len(r["courses"]) == min(6, n)
def t9():
    top = lambda f, n: list(analyse(f)["domain_ranking"])[:n]
    assert top(A, 1) == ["Data Science"] and top(C, 1) == ["Cybersecurity"]
    assert "Full Stack Development" in top(B, 2)
def t10():
    try: analyse(A, "Quantum Gardening"); assert False
    except ValueError: pass
def t11():
    c = app.test_client()
    r = c.post("/analyse", data={"resume": (open(B, "rb"), "b.pdf"), "domain": "Full Stack
Development"}, content_type="multipart/form-data")
    assert r.status_code == 200 and r.get_json()["target_domain"] == "Full Stack Development"
def t12():
    c = app.test_client()
    r = c.post("/analyse", data={"resume": (io.BytesIO(b"x"), "x.exe")},
content_type="multipart/form-data")
    assert r.status_code == 400
    assert c.post("/analyse", data={}, content_type="multipart/form-data").status_code == 400
def t13():
    missing = {s for d in DOMAINS.values() for k in ("core", "optional") for s in d[k]} -
set(RESOURCES)
    assert not missing, missing

tests = [("T1","Parse PDF resume and extract e-mail",t1),("T2","Parse DOCX resume and extract
name",t2),
 ("T3","Reject unsupported .txt upload",t3),("T4","Handle corrupt PDF without crashing",t4),
 ("T5","Reject DOCX with no readable text",t5),("T6","Analyse against each of the four domains",t6),
 ("T7","Match score stays within 0-100 (12 combinations)",t7),("T8","Recommendations generated for
missing skills",t8),
 ("T9","Intended domain ranked first (A, C) or in top two (B)",t9),("T10","Reject unknown domain
name",t10),
 ("T11","Flask /analyse endpoint returns 200 with JSON",t11),("T12","Flask rejects bad extension /
missing file (HTTP 400)",t12),
 ("T13","Every taxonomy skill has a learning resource",t13)]
analyse(A)  # warm-up (loads NLTK corpora once)
for tid, desc, fn in tests: case(tid, desc, fn)
times = []
for _ in range(10):
    for f in (A, B, C): times.append(analyse(f)["elapsed_ms"])
summary = {"tests": results, "passed": sum(r[2] == "Pass" for r in results), "total": len(results),
           "mean_ms": round(statistics.mean(times), 1), "max_ms": round(max(times), 1), "runs":
len(times)}
json.dump(summary, open("../test_results.json", "w"), indent=1)
for r in results: print(r)
print(summary["passed"], "/", summary["total"], "mean ms", summary["mean_ms"], "max",
summary["max_ms"])
```

5.4 RESULTS

Test Setup

The pipeline was run on three fictional sample resumes created for this report: two in DOCX format and one in PDF format (Table 5.1). They were written to represent a data-science student, a full-stack developer and a cybersecurity beginner. All text was extracted from the

actual files by the implemented parser. Experiments ran on a CPU-only machine without a GPU.

Table 5.1: Sample resumes used for the demonstration

Resume	Format	Words	Sections detected	Declared career interest
Resume A	DOCX	98	certifications, education, experience, projects, skills, summary	Data Science
Resume B	PDF	90	certifications, education, experience, projects, skills, summary	Full Stack Development
Resume C	DOCX	43	education, projects, skills	Cybersecurity

Extracted Skills

Table 5.2 lists the skills that the extraction module found. Spelling variants were resolved correctly: for example "Scikit-learn" and "Matplotlib" in Resume A were mapped to scikit-learn and data visualization, "Node.js, Express.js, HTML5, CSS3 and REST APIs" in Resume B were mapped to their canonical names, and "TCP/IP networking" and "CTF practice" in Resume C were mapped to networking and ethical hacking.

Table 5.2: Skills extracted from each resume

Resume	Count	Skills found
Resume A	9	data cleaning, data visualization, machine learning, numpy, pandas, python, scikit-learn, sql, statistics
Resume B	12	bootstrap, css, express, git, html, javascript, jwt, mongodb, node.js, react, responsive design, rest api
Resume C	7	cryptography, ethical hacking, linux, networking, nmap, python, wireshark

Domain Suitability and Skill Match

Figure 5.1 compares the score of every resume against every domain, and Table 5.3 gives the components of the score for the domain that the system suggested. For Resume A the suggested domain is Data Science with a score of 54.9; all nine core skills are present and the score is held down only by the eight missing optional skills. Resume C receives 29.4 for Cybersecurity because six of the nine core skills are missing. The ranking separates the domains clearly: unrelated domains score below 6 out of 100 for all three resumes.

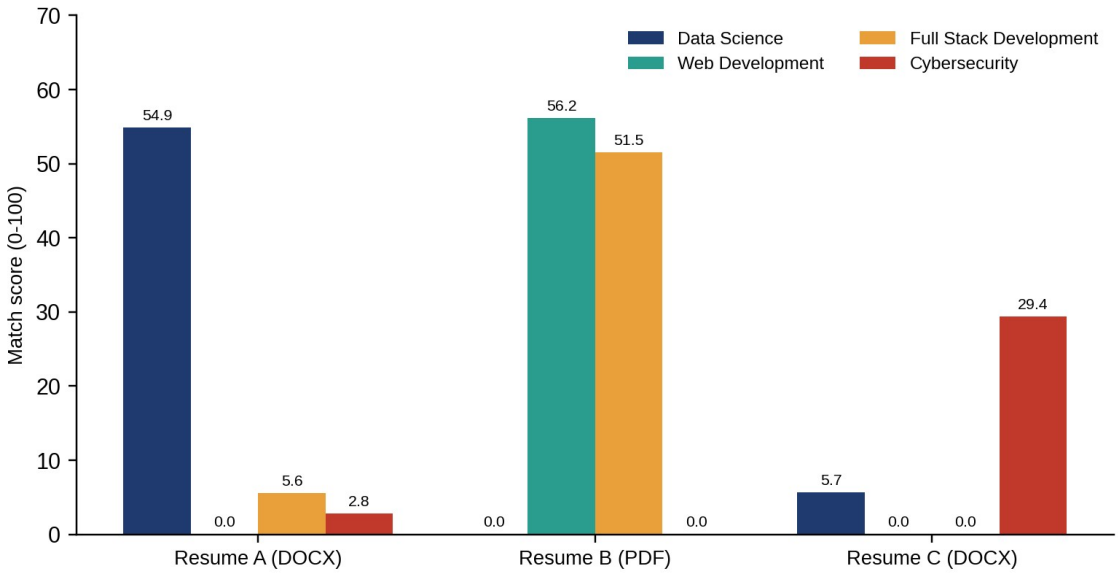


Figure 5.1: Match score of each sample resume against each domain

Table 5.3: Score components for the suggested domain

Resume	Suggested domain	Score	Skill coverage	TF-IDF cosine	Matched	Missing core	Missing optional
Resume A	Data Science	54.9	69.2 %	0.213	9	0	8
Resume B	Web Development	56.2	72.7 %	0.177	8	0	6
Resume C	Cybersecurity	29.4	38.5 %	0.083	7	6	4

Resume B illustrates the overlap between related domains. It was written as a full-stack profile, yet Web Development ranks first (56.2) and Full Stack Development second (51.5), because the Web Development profile has fewer core skills and the resume covers them completely. When the user selects Full Stack Development as the target, the score is 51.5 with skill coverage 67.9 % and one missing core skill (sql). This is a useful result for the user, since the dashboard tells the candidate that adding SQL would complete the core stack. It also shows why the full ranking, and not only the top domain, is displayed.

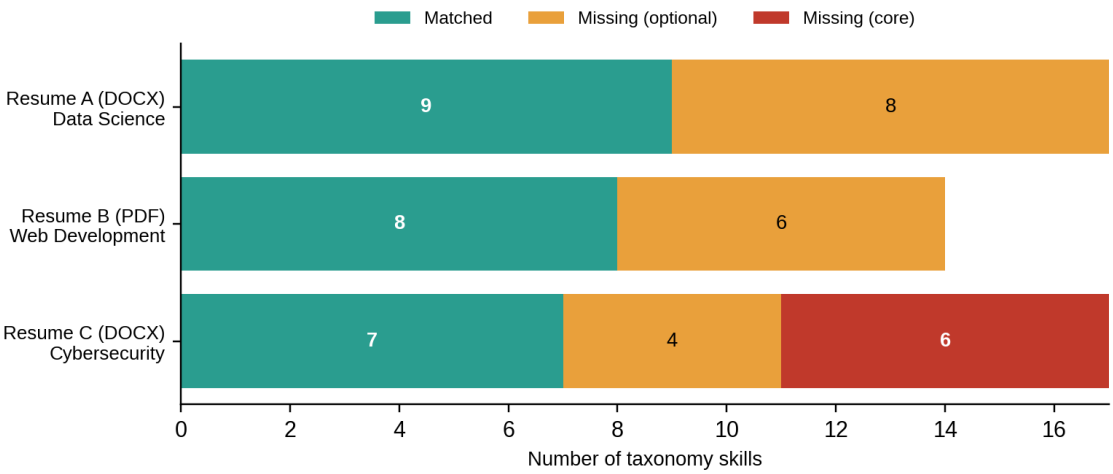


Figure 5.2: Matched and missing skills for the suggested domain

Dashboard Output

Figures 5.3 to 5.5 show the dashboard layout generated from the JSON output of the implemented pipeline for each resume: the score card, the domain ranking, the matched and missing skills, the recommended learning resources and the resume tips.

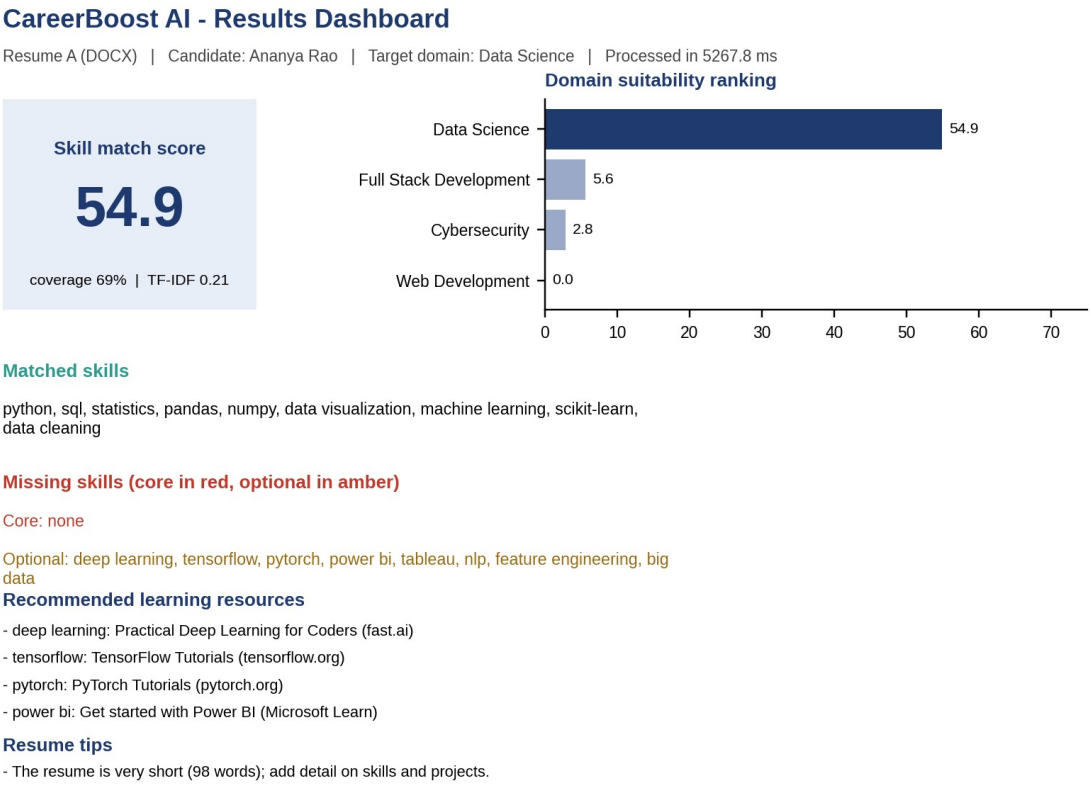


Figure 5.3: Dashboard output for Resume A (Data Science)

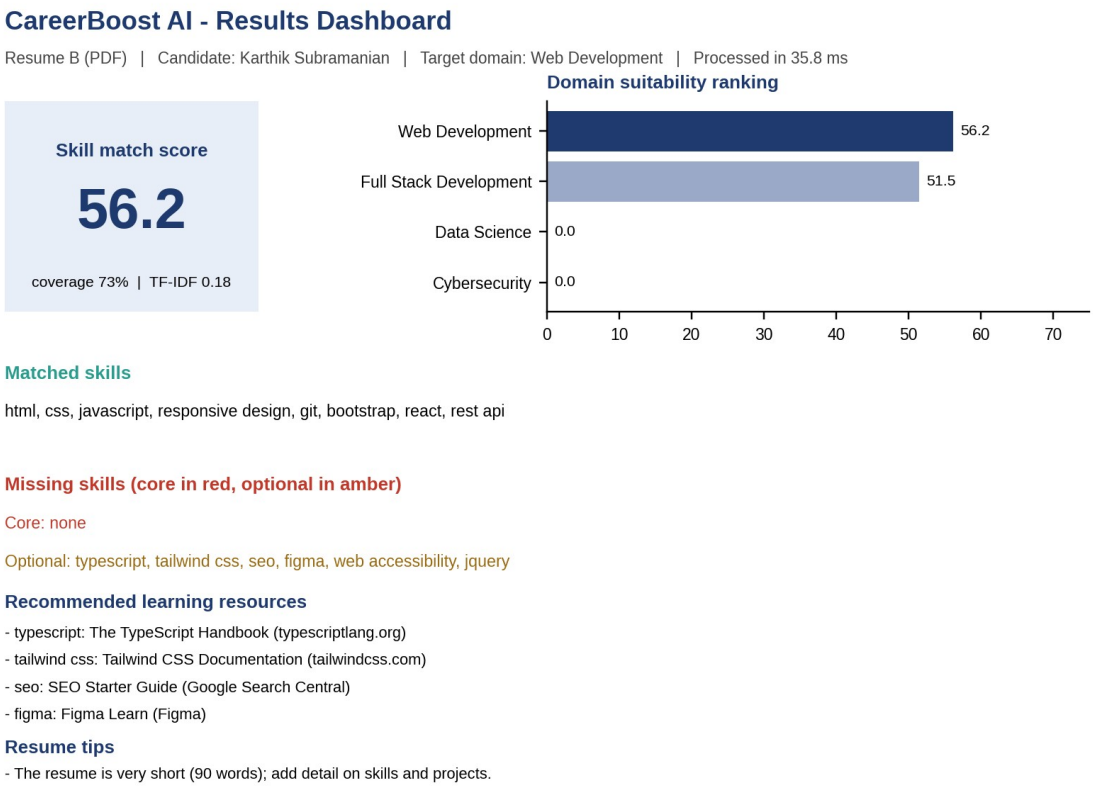


Figure 5.4: Dashboard output for Resume B (Web Development ranked first)

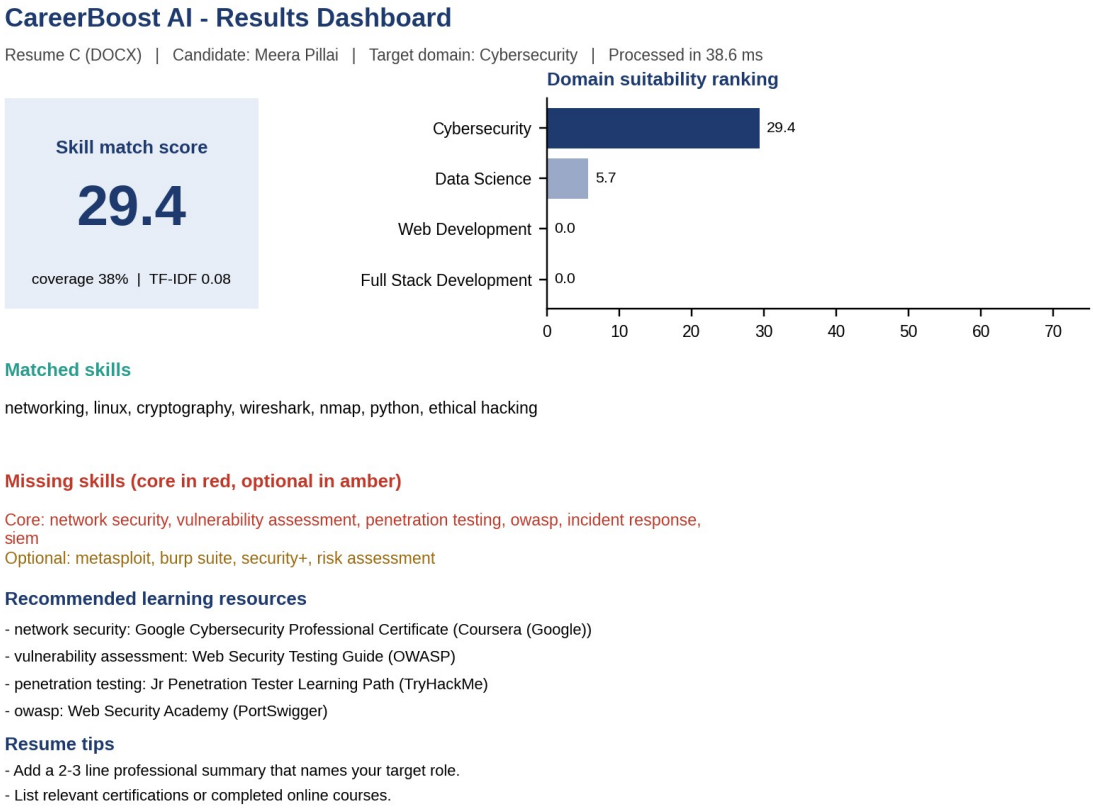


Figure 5.5: Dashboard output for Resume C (Cybersecurity)

Recommendations Generated

For Resume C, the recommender returned the six learning resources in Table 5.4, ordered with the missing core skills first. Resume tips for the same resume were: (1) Add a 2-3 line professional summary that names your target role. (2) List relevant certifications or completed online courses. (3) Include GitHub and LinkedIn profile links. (4) Quantify project results (accuracy, users, speed-up, dataset size). (5) The resume is very short (43 words); add detail on skills and projects.

Table 5.4: Learning resources recommended for Resume C

Missing skill	Resource	Provider	Type
network security	Google Cybersecurity Professional Certificate	Coursera (Google)	Certification
vulnerability assessment	Web Security Testing Guide	OWASP	Free guide
penetration testing	Jr Penetration Tester Learning Path	TryHackMe	Freemium path
owasp	Web Security Academy	PortSwigger	Free course
incident response	Computer Security Incident Handling Guide (SP 800-61)	NIST	Free guide
siem	Splunk Fundamentals 1	Splunk Training	Free course

Functional Testing and Response Time

Table 5.5 lists the 13 functional test cases and their outcomes. 13 of 13 passed. The tests cover file parsing, input validation, scoring limits, recommendation generation, domain ranking and the Flask endpoint.

Table 5.5: Functional test cases and results

ID	Test case	Result
T1	Parse PDF resume and extract e-mail	Pass
T2	Parse DOCX resume and extract name	Pass
T3	Reject unsupported .txt upload	Pass
T4	Handle corrupt PDF without crashing	Pass
T5	Reject DOCX with no readable text	Pass
T6	Analyse against each of the four domains	Pass
T7	Match score stays within 0-100 (12 combinations)	Pass
T8	Recommendations generated for missing skills	Pass
T9	Intended domain ranked first (A, C) or in top two (B)	Pass
T10	Reject unknown domain name	Pass
T11	Flask /analyse endpoint returns 200 with JSON	Pass
T12	Flask rejects bad extension / missing file (HTTP 400)	Pass

ID	Test case	Result
T13	Every taxonomy skill has a learning resource	Pass

Response time was measured over 30 analyses of the three sample resumes after the language resources had been loaded. The mean processing time was 24.7 ms and the maximum was 50.1 ms. The first request after start-up takes about four seconds because the NLTK language resources are loaded once.

Discussion

The results show that the pipeline reads both file formats correctly, rejects invalid input safely, recognises skills despite spelling variants and gives scores that behave sensibly: a resume that covers its domain well scores higher than one with many missing core skills, and unrelated domains receive very low scores. The explanation attached to every score, in the form of matched skills, missing skills and resources, is the main practical advantage over a bare keyword score. The three sample resumes were written by the project team, so these results confirm correct operation and do not measure accuracy on real-world resumes. A labelled dataset and a larger user study are listed under future work.

REFERENCES

[1] K. L. Abhishek, M. Niranjanamurthy, et al., "Developing an Intelligent Resume Screening Tool With AI-Driven Analysis and Recommendation Features," *Applied AI Letters*, vol. 6, no. 2, 2025.

[2] D. Lavi, V. Medentsiy, and D. Graus, "conSultantBERT: Fine-tuned Siamese Sentence-BERT for Matching Jobs and Job Seekers," in *Proc. RecSys in HR 2021*, 2021, arXiv:2109.06501.

[3] R. Soni, B. Suleiman, and S. Singh, "Improving Graduate Outcomes by Identifying Skills Gaps and Recommending Courses Based on Career Interests," arXiv:2511.09819, Nov. 2025.

[4] A. Parab, S. Palkar, S. Maurya, and S. Balpande, "An Intelligent Career Counselling Bot: A System for Counselling," *Int. Res. J. Eng. Technol. (IRJET)*, vol. 4, no. 3, pp. 2325–2330, Mar. 2017.

[5] A. Sabarirajan, G. Chandramowleeswaran, D. Thamizharasi, S. Susendiran, G. Priyadharshini, and S. H. Abbas, "Intelligent AI-Based Resume Screening and Ranking Framework for Unbiased and Scalable Recruitment Automation," in *Proc. 1st Int. Conf. on Research and Development in Information, Communication, and Computing Technologies (ICRDICCT'25)*, SciTePress, 2025, pp. 158–164, doi: 10.5220/0013859500004919.

[6] P. Tijare, M. Waseem, M. A. Sherani, K. S. K. S. Krishna, and P. Kavitha, "A smart resume screening tool for customized shortlisting," *ITM Web of Conferences*, vol. 56, art. 04001, 2023, doi: 10.1051/itmconf/20235604001.

[7] C. Daryani, G. S. Chhabra, H. Patel, I. K. Chhabra, and R. Patel, "An Automated Resume Screening System Using Natural Language Processing and Similarity," *Topics in Intelligent Computing and Industry Design (ICID)*, vol. 2, no. 2, pp. 99–103, 2020, doi: 10.26480/etit.02.2020.99.103.

[8] V. Lai, K. J. Shim, R. J. Oentaryo, P. K. Prasetyo, C. Vu, E.-P. Lim, and D. Lo, "CareerMapper: An Automated Resume Evaluation Tool," arXiv:1611.05339, 2016.

[9] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. 2019 Conf. on Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. on Natural Language Processing (EMNLP-IJCNLP)* 2019, pp. 3982–3992.

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[11] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[12] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[13] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[14] K. Spärck Jones, "A statistical interpretation of term specificity and its application in retrieval," *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.

[15] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.

[16] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. Sebastopol, CA, USA: O'Reilly Media, 2009.

[17] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-strength Natural Language Processing in Python," Zenodo, 2020, doi: 10.5281/zenodo.1212303.

[18] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[19] M. Grinberg, *Flask Web Development*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.

[20] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.